

# Modelización Jerárquica y Simulación Estocástica de Estrategias de Parada en Boxes de Fórmula 1 (2018-2024)

Trabajo de Fin de Grado — Código fuente en R

María Varela Oyola

2026-06-01

## Tabla de contenidos

|        |                                                                                  |    |
|--------|----------------------------------------------------------------------------------|----|
| 0.1    | Introducción . . . . .                                                           | 2  |
| 0.2    | Preparación y Limpieza de Datos . . . . .                                        | 3  |
| 0.2.1  | Carga de Datos y Estandarización de Cabeceras . . . . .                          | 3  |
| 0.2.2  | Estructuración de Variables Jerárquicas y Temporales . . . . .                   | 3  |
| 0.2.3  | Procesamiento del Entorno: Clima y Topografía . . . . .                          | 4  |
| 0.2.4  | Aislamiento de Métricas de Recompensa y Control de Fuga (Data Leakage) . . . . . | 5  |
| 0.2.5  | Análisis de Supervivencia: Neumáticos y Censura Estocástica . . . . .            | 6  |
| 0.2.6  | Bifurcación Analítica: Generación del Entorno Estricto Ex-Ante . . . . .         | 6  |
| 0.3    | Selección de Variables y Diagnóstico de Colinealidad . . . . .                   | 7  |
| 0.3.1  | Justificación de la Variable Objetivo: Correlación de Spearman . . . . .         | 7  |
| 0.3.2  | Análisis VIF (I): Fase 1 . . . . .                                               | 8  |
| 0.3.3  | Análisis VIF (II): Fase 2 (Colinealidad Estructural) . . . . .                   | 8  |
| 0.3.4  | Análisis VIF (III): Fase 3 (Centrado de Medias) . . . . .                        | 9  |
| 0.3.5  | Lasso (I): Estandarización L1 . . . . .                                          | 10 |
| 0.3.6  | Lasso (II): Motor 1 (Degradación) . . . . .                                      | 10 |
| 0.3.7  | Lasso (III): Motor 2 (Vida Útil) . . . . .                                       | 11 |
| 0.3.8  | Geometría Lasso vs Ridge (I): Entorno Gráfico . . . . .                          | 12 |
| 0.3.9  | Geometría Lasso vs Ridge (II): Lasso . . . . .                                   | 13 |
| 0.3.10 | Geometría Lasso vs Ridge (III): Ridge . . . . .                                  | 14 |
| 0.4    | Modelización: Modelos Lineales Mixtos (LMM) . . . . .                            | 15 |
| 0.4.1  | Estimación Base OLS: Motor 1 . . . . .                                           | 15 |
| 0.4.2  | Estimación Base OLS: Motor 2 . . . . .                                           | 15 |
| 0.4.3  | Entrenamiento LMM: Motor 1 (Degradación) . . . . .                               | 16 |

|        |                                                                              |    |
|--------|------------------------------------------------------------------------------|----|
| 0.4.4  | Entrenamiento LMM: Motor 2 (Vida Útil) . . . . .                             | 16 |
| 0.4.5  | Efectos Aleatorios (Lollipop Plot) . . . . .                                 | 17 |
| 0.4.6  | Efectos Fijos (Forest Plot) . . . . .                                        | 19 |
| 0.4.7  | Test de Razón de Verosimilitud (LRT): Motor 1 . . . . .                      | 20 |
| 0.4.8  | Test de Razón de Verosimilitud (LRT): Motor 2 . . . . .                      | 21 |
| 0.4.9  | Validación Estructural AIC/BIC: Tabulación de Métricas . . . . .             | 21 |
| 0.4.10 | Corrección ICC: Singularidad Escalar en Motor 1 . . . . .                    | 23 |
| 0.4.11 | Extracción de Pseudo R-Cuadrado de Nakagawa . . . . .                        | 23 |
| 0.4.12 | Gráfico de Varianza Explicada (ICC) . . . . .                                | 24 |
| 0.4.13 | Validación de Supuestos Matemáticos . . . . .                                | 25 |
| 0.4.14 | Predicciones Marginales Termodinámicas . . . . .                             | 27 |
| 0.5    | Simulación de Estrategia de Carrera (Monte Carlo) . . . . .                  | 29 |
| 0.5.1  | Simulación (I): Carga y Entrenamiento Operativo . . . . .                    | 29 |
| 0.5.2  | Simulación (II): Diccionario Topográfico y Logístico . . . . .               | 30 |
| 0.5.3  | Simulación (III): Consola del Muro . . . . .                                 | 31 |
| 0.5.4  | Simulación (IV): Generador Heurístico . . . . .                              | 32 |
| 0.5.5  | Simulación (V): Motor Estocástico de Caos . . . . .                          | 33 |
| 0.5.6  | Simulación (VI): Integración de Monte Carlo Vectorizada . . . . .            | 35 |
| 0.6    | Análisis Estocástico de Resultados . . . . .                                 | 36 |
| 0.6.1  | Análisis Estocástico (I): Matriz de Decisión . . . . .                       | 36 |
| 0.6.2  | Análisis Estocástico (II): Representación Densidad KDE . . . . .             | 37 |
| 0.6.3  | Análisis Estocástico (III): Gráfico de Convergencia Asintótica . . . . .     | 39 |
| 0.7    | Optimización Prescriptiva . . . . .                                          | 41 |
| 0.7.1  | Optimización Prescriptiva (I): Heurística de Transferencia . . . . .         | 41 |
| 0.7.2  | Optimización Prescriptiva (II): Evaluación Multicriterio . . . . .           | 42 |
| 0.7.3  | Optimización Prescriptiva (III): Frontera de Eficiencia de Pareto . . . . .  | 44 |
| 0.8    | Auditoría Final del Modelo . . . . .                                         | 46 |
| 0.8.1  | Auditoría Final (I): Simulación Extendida del Optimo (N=10.000) . . . . .    | 46 |
| 0.8.2  | Auditoría Final (II): Gráfico Comparativo de Reducción de Varianza . . . . . | 48 |
| 0.8.3  | Auditoría Final (III): Diagrama de Prescripción Táctica (Gantt) . . . . .    | 49 |

## 0.1 Introducción

Este documento recoge, organizado por bloques temáticos y con una breve explicación antes de cada fragmento, el código fuente en R desarrollado para el Trabajo de Fin de Grado. El análisis parte de un registro histórico de paradas en boxes de Fórmula 1 (temporadas 2018-2024) y se estructura en cuatro grandes fases:

1. **Preparación de datos:** limpieza y construcción de variables jerárquicas, temporales y ambientales.
2. **Selección de variables:** justificación de la variable objetivo, diagnóstico de colinealidad (VIF) y regularización (Lasso).

3. **Modelización:** ajuste de Modelos Lineales Mixtos (LMM) con efectos aleatorios por circuito, piloto y constructor, y su validación frente a modelos OLS de referencia (LRT, AIC/BIC, ICC,  $R^2$  de Nakagawa).
4. **Simulación y optimización:** un simulador de Monte Carlo de estrategias de parada en boxes, alimentado por los modelos mixtos anteriores, junto con una fase de optimización prescriptiva y una auditoría final del resultado.

## 0.2 Preparación y Limpieza de Datos

### 0.2.1 Carga de Datos y Estandarización de Cabeceras

Se importa el dataset de paradas en boxes de Fórmula 1 (temporadas 2018–2024) desde el archivo CSV y se renombran sus 29 columnas originales a nombres descriptivos en español, que se utilizarán de forma consistente durante todo el análisis.

```
# --- Carga de Datos y Estandarizacion de Cabeceras ---
f1 = read.csv2("f1_pitstops_2018_2024.csv", sep = ",", header = T)

colnames(f1) = c("Temporada", "Ronda", "Circuito", "Piloto", "Constructor",
  "Vueltas", "Posicion", "Total_Paradas_Pit",
  ↪ "Tiempo_Medio_Parada_Pit",
  "Nombre_Carrera", "Fecha", "Hora_Carrera", "Ubicacion",
  ↪ "Pais",
  "Temperatura_Aire_C", "Temperatura_Pista_C",
  ↪ "Humedad_Porcentaje",
  "Velocidad_Viento_KMH", "Variacion_Tiempo_Vuelta",
  ↪ "Total_Paradas_Pit_Carrera",
  "Agresividad_Uso_Neumaticos", "Intentos_Vuelta_Rapida",
  ↪ "Cambios_Posicion",
  "Puntuacion_Agresividad_Piloto", "Abreviatura", "Stint",
  "Compuesto_Neumatico", "Duracion_Stint", "Vuelta_Parada",
  ↪ "Tiempo_Parada")
```

### 0.2.2 Estructuración de Variables Jerárquicas y Temporales

Se convierte la temporada a variable numérica y a factor, y se calcula la ronda normalizada dentro de cada temporada para derivar la fase del campeonato (Inicio, Mitad o Final). Circuito, Piloto, Constructor y Abreviatura se transforman en factores, ya que actuarán como niveles jerárquicos en los modelos mixtos posteriores.

```
# --- Estructuración de Variables Jerárquicas y Temporales ---
f1$Temporada = as.numeric(f1$Temporada)
f1$Temporada_factor = as.factor(f1$Temporada)

total_rondas = ave(f1$Ronda, f1$Temporada, FUN = max)
f1$Ronda_normalizada = f1$Ronda / total_rondas
f1$Fase_Campeonato = as.factor(ifelse(f1$Ronda_normalizada <= 0.33, "Inicio",
↪
                                     ifelse(f1$Ronda_normalizada <= 0.66,
↪      "Mitad", "Final"))))

f1$Circuito = as.factor(f1$Circuito)
f1$Piloto = as.factor(f1$Piloto)
f1$Constructor = as.factor(f1$Constructor)
f1$Abreviatura = as.factor(f1$Abreviatura)
```

### 0.2.3 Procesamiento del Entorno: Clima y Topografía

Se procesa la fecha de carrera para extraer el mes y la estación del año, se transforma la hora de carrera en segundos desde medianoche, se armonizan los nombres de país (por ejemplo, USA → United States) y se estandarizan (z-score) la temperatura de pista, la temperatura del aire, la humedad y la velocidad del viento. También se genera el término cuadrático de la temperatura de pista, que se empleará más adelante para modelar relaciones no lineales.

```
# --- Procesamiento del Entorno: Clima y Topografía ---
f1$Fecha = as.Date(f1$Fecha, format = "%d-%m-%Y")
f1$Mes = as.numeric(format(f1$Fecha, "%m"))
f1$Estacion = as.factor(ifelse(f1$Mes %in% c(12, 1, 2), "Invierno",
                               ifelse(f1$Mes %in% c(3, 4, 5), "Primavera",
                               ifelse(f1$Mes %in% c(6, 7, 8),
↪      "Verano", "Otono"))))

f1$Hora_Carrera_Clean = as.POSIXct(f1$Hora_Carrera, format = "%H:%M:%S", tz =
↪      "UTC")
f1$Hora_segundos = as.numeric(f1$Hora_Carrera_Clean -
↪      as.POSIXct(format(f1$Hora_Carrera_Clean, "%Y-%m-%d")))

f1$Pais[f1$Pais == "USA"] = "United States"
f1$Pais[f1$Pais == "UK"] = "United Kingdom"
f1$Pais[f1$Pais == "UAE"] = "United Arab Emirates"
f1$Pais[f1$Pais == ""] = NA
```

```
f1$Pais = as.factor(f1$Pais)

f1$Temperatura_Pista_C = as.numeric(f1$Temperatura_Pista_C)
f1$Temperatura_Pista_scaled = scale(f1$Temperatura_Pista_C)
f1$Temperatura_Pista_C2 = f1$Temperatura_Pista_C^2

f1$Temperatura_Aire_scaled = scale(as.numeric(f1$Temperatura_Aire_C))
f1$Humedad_scaled = scale(as.numeric(f1$Humedad_Porcentaje))
f1$Viento_scaled = scale(as.numeric(f1$Velocidad_Viento_KMH))
```

#### 0.2.4 Aislamiento de Métricas de Recompensa y Control de Fuga (Data Leakage)

Se calculan los puntos obtenidos por cada piloto según su posición final (sistema de puntuación oficial de la F1) y la proporción de vueltas completadas respecto al total de la carrera. Además, se estandarizan la variación del tiempo de vuelta y los cambios de posición, y se crea una variable indicadora de si el piloto intentó la vuelta rápida. Este bloque aísla explícitamente las métricas de “resultado” para evitar que se filtren como predictores (data leakage) en fases posteriores.

```
# --- Aislamiento de Metricas de Recompensa y Control de Fuga (Data Leakage)
↪ ---
f1$Posicion_Num = as.numeric(as.character(f1$Posicion))
puntos_f1 = c(25, 18, 15, 12, 10, 8, 6, 4, 2, 1)
f1$Puntos_Finales = ifelse(f1$Posicion_Num <= 10, puntos_f1[f1$Posicion_Num],
↪ 0)
f1$Puntos_Finales[is.na(f1$Puntos_Finales)] = 0

total_vueltas_carrera = ave(f1$Vueltas, f1$Temporada, f1$Ronda, FUN =
↪ function(x) {
  if(all(is.na(x))) NA else max(x, na.rm = TRUE) })
f1$Proporcion_Vueltas = f1$Vueltas / total_vueltas_carrera

f1$Variacion_Tiempo_scaled = scale(as.numeric(f1$Variacion_Tiempo_Vuelta))
f1$Cambios_Posicion_scaled = scale(as.numeric(f1$Cambios_Posicion))
f1$Intento_VR_Dummy = as.factor(ifelse(as.numeric(f1$Intentos_Vuelta_Rapida)
↪ > 0, 1, 0))
```

### 0.2.5 Análisis de Supervivencia: Neumáticos y Censura Estocástica

Se homogeneizan los compuestos de neumático antiguos (Hypersoft, Ultrasoft, Supersoft) bajo la etiqueta “SOFT” y se fija “MEDIUM” como nivel de referencia del factor. Se identifican los stints que terminan sin una parada registrada —es decir, censurados, porque llegan hasta el final de la carrera— y se imputa su vuelta de finalización con el número total de vueltas. Por último, se limpian y estandarizan los tiempos de parada en boxes.

```
# --- Analisis de Supervivencia: Neumaticos y Censura Estocastica ---
f1$Compuesto_Neumatico = toupper(as.character(f1$Compuesto_Neumatico))
f1$Compuesto_Neumatico[f1$Compuesto_Neumatico %in% c("HYPERSOFT",
  ↪ "ULTRASOFT", "SUPERSOFT")] = "SOFT"
f1$Compuesto_Neumatico[f1$Compuesto_Neumatico %in% c("UNKNOWN", "")] = NA
f1$Compuesto_Neumatico = as.factor(f1$Compuesto_Neumatico)
f1$Compuesto_Neumatico = relevel(f1$Compuesto_Neumatico, ref = "MEDIUM")

f1$Vuelta_Parada_Real = as.numeric(f1$Vuelta_Parada)
f1$Stint_Final = ifelse(is.na(f1$Vuelta_Parada_Real), 1, 0)
f1$Vuelta_Parada_Real[is.na(f1$Vuelta_Parada_Real)] =
  ↪ f1$Vueltas[is.na(f1$Vuelta_Parada_Real)]

f1$Tiempo_Parada = suppressWarnings(as.numeric(f1$Tiempo_Parada))
f1$Tiempo_Parada[is.na(f1$Tiempo_Parada)] = 0
f1$Tiempo_Parada_scaled = scale(f1$Tiempo_Parada)

f1$Tiempo_Medio_Parada_Pit =
  ↪ suppressWarnings(as.numeric(f1$Tiempo_Medio_Parada_Pit))
f1$Tiempo_Medio_Parada_Pit[is.na(f1$Tiempo_Medio_Parada_Pit)] = 0
f1$Tiempo_Medio_Parada_scaled = scale(f1$Tiempo_Medio_Parada_Pit)
```

### 0.2.6 Bifurcación Analítica: Generación del Entorno Estricto Ex-Ante

Se construye un subconjunto de variables que solo contiene información disponible antes de que finalice cada stint (un entorno “ex-ante” estricto, sin fuga de información), se eliminan los registros incompletos y se guardan en disco tanto el dataset completo como este subconjunto, en formato .rds, para poder reutilizarlos sin repetir todo el preprocesado.

```
# --- Bifurcacion Analitica: Generacion del Entorno Estricto Ex-Ante ---
variables_motor = c("Temporada", "Circuito", "Piloto", "Constructor",
  "Temperatura_Pista_C", "Temperatura_Pista_C2",
  "Stint", "Compuesto_Neumatico", "Duracion_Stint",
```

```

        "Variacion_Tiempo_Vuelta", "Tiempo_Parada",
        ↪  "Stint_Final")

f1_motor_estrategia = f1[, variables_motor]
f1_motor_estrategia = na.omit(f1_motor_estrategia)

saveRDS(f1, file = "f1_limpia.rds")
saveRDS(f1_motor_estrategia, file = "f1_motor_estrategia.rds")

```

## 0.3 Selección de Variables y Diagnóstico de Colinealidad

### 0.3.1 Justificación de la Variable Objetivo: Correlación de Spearman

Se realiza un test de correlación de Spearman entre la variación del tiempo de vuelta y los puntos finales obtenidos, como justificación empírica de que el ritmo de carrera está relacionado con el rendimiento competitivo y, por tanto, es una variable objetivo razonable para el resto del análisis.

```

# --- Justificacion de la Variable Objetivo: Correlacion de Spearman ---
f1$Variacion_Tiempo_Vuelta =
  ↪  as.numeric(as.character(f1$Variacion_Tiempo_Vuelta))

datos_correlacion = na.omit(f1[, c("Variacion_Tiempo_Vuelta",
  ↪  "Puntos_Finales"))])
test_objetivo = cor.test(datos_correlacion$Variacion_Tiempo_Vuelta,
                        datos_correlacion$Puntos_Finales,
                        method = "spearman")

print(test_objetivo)

```

Spearman's rank correlation rho

```

data:  datos_correlacion$Variacion_Tiempo_Vuelta and
  ↪  datos_correlacion$Puntos_Finales
S = 6.7381e+10, p-value = 7.062e-14
alternative hypothesis: true rho is not equal to 0
sample estimates:
      rho
-0.08814276

```

### 0.3.2 Análisis VIF (I): Fase 1

Se ajusta un primer modelo lineal mixto (lmer) de la variación del tiempo de vuelta con efectos aleatorios por circuito, piloto y constructor, incluyendo la temperatura del aire y la temperatura de pista como predictores independientes. A continuación se calcula el Factor de Inflación de la Varianza Generalizado (GVIF), ajustado por los grados de libertad de cada variable, para detectar colinealidad entre predictores.

```
# --- Analisis VIF (I): Fase 1 ---
f1$Temperatura_Aire_C = as.numeric(as.character(f1$Temperatura_Aire_C))
f1$Temperatura_Pista_C = as.numeric(as.character(f1$Temperatura_Pista_C))
f1$Circuito = as.factor(f1$Circuito)
f1$Piloto = as.factor(f1$Piloto)
f1$Constructor = as.factor(f1$Constructor)

library(lme4)
library(car)

modelo_fase1 = lmer(Variacion_Tiempo_Vuelta ~ Compuesto_Neumatico +
  ↪ Duracion_Stint +
  ↪ Temperatura_Aire_C + Temperatura_Pista_C +
  ↪ Humedad_scaled +
  ↪ Viento_scaled + (1 | Circuito) + (1 | Piloto) + (1 |
  ↪ Constructor),
  data = f1, control = lmerControl(optimizer = "bobyqa"))

vif_fase1 = vif(modelo_fase1)
vif_adj_fase1 = vif_fase1[, "GVIF^(1/(2*Df))"]^2
print(vif_adj_fase1)
```

| Compuesto_Neumatico   | Duracion_Stint | Temperatura_Aire_C |
|-----------------------|----------------|--------------------|
| ↪ Temperatura_Pista_C |                |                    |
| 1.055920              | 1.191778       | 282.136820         |
| ↪ 303.885799          |                |                    |
| Humedad_scaled        | Viento_scaled  |                    |
| 2.856632              | 1.097983       |                    |

### 0.3.3 Análisis VIF (II): Fase 2 (Colinealidad Estructural)

Se repite el modelo anterior sustituyendo las dos temperaturas por la temperatura de pista junto con su término cuadrático, y se recalcula el VIF ajustado. Este paso evidencia la colinea-



lidad estructural que aparece de forma esperada al incluir una variable y su propio cuadrado en el mismo modelo.

```
# --- Analisis VIF (II): Fase 2 (Colinealidad Estructural) ---
f1$Temperatura_Pista_C2 = f1$Temperatura_Pista_C^2

modelo_fase2 = lmer(Variacion_Tiempo_Vuelta ~ Compuesto_Neumatico +
  ↪ Duracion_Stint +
  ↪ Temperatura_Pista_C + Temperatura_Pista_C2 +
  ↪ Humedad_scaled +
  ↪ Viento_scaled + (1 | Circuito) + (1 | Piloto) + (1 |
  ↪ Constructor),
  data = f1, control = lmerControl(optimizer = "bobyqa"))

vif_fase2 = vif(modelo_fase2)
vif_adj_fase2 = vif_fase2[, "GVIF^(1/(2*Df))"]^2
print(vif_adj_fase2)
```

|                      |                |                     |
|----------------------|----------------|---------------------|
| Compuesto_Neumatico  | Duracion_Stint | Temperatura_Pista_C |
| 1.054237             | 1.183708       | 21.079997           |
| Temperatura_Pista_C2 | Humedad_scaled | Viento_scaled       |
| 20.794792            | 1.418514       | 1.056298            |

### 0.3.4 Análisis VIF (III): Fase 3 (Centrado de Medias)

Se centra la temperatura de pista en su media antes de elevarla al cuadrado (Temp\_Pista\_Centrada y Temp\_Pista\_Centrada2) y se vuelve a ajustar el modelo mixto con estos términos centrados. El VIF ajustado resultante permite comprobar si el centrado mitiga la colinealidad estructural detectada en la fase anterior.

```
# --- Analisis VIF (III): Fase 3 (Centrado de Medias) ---
f1$Temp_Pista_Centrada = f1$Temperatura_Pista_C -
  ↪ mean(f1$Temperatura_Pista_C, na.rm = TRUE)
f1$Temp_Pista_Centrada2 = f1$Temp_Pista_Centrada^2

modelo_fase3 = lmer(Variacion_Tiempo_Vuelta ~ Compuesto_Neumatico +
  ↪ Duracion_Stint +
  ↪ Temp_Pista_Centrada + Temp_Pista_Centrada2 +
  ↪ Humedad_scaled +
  ↪ Viento_scaled + (1 | Circuito) + (1 | Piloto) + (1 |
  ↪ Constructor),
  data = f1, control = lmerControl(optimizer = "bobyqa"))
```

```
vif_fase3 = vif(modelo_fase3)
vif_adj_fase3 = vif_fase3[, "GVIF^(1/(2*Df))"]^2
print(vif_adj_fase3)
```

|                      |                |                     |
|----------------------|----------------|---------------------|
| Compuesto_Neumatico  | Duracion_Stint | Temp_Pista_Centrada |
| 1.054237             | 1.183708       | 1.536933            |
| Temp_Pista_Centrada2 | Humedad_scaled | Viento_scaled       |
| 1.110572             | 1.418514       | 1.056298            |

### 0.3.5 Lasso (I): Estandarización L1

Se reconvierten a numéricas y se reescalan las variables de agresividad del piloto, humedad, viento y temperatura de pista, preparando los datos para el proceso de selección de variables mediante regresión Lasso en los dos “motores” del modelo (degradación y vida útil del neumático).

```
# --- Lasso (I): Estandarizacion L1 ---
library(glmnet)

f1$Puntuacion_Agresividad_Piloto =
  ↪ as.numeric(as.character(f1$Puntuacion_Agresividad_Piloto))
f1$Humedad_Porcentaje = as.numeric(as.character(f1$Humedad_Porcentaje))
f1$Velocidad_Viento_KMH = as.numeric(as.character(f1$Velocidad_Viento_KMH))
f1$Temperatura_Pista_C = as.numeric(as.character(f1$Temperatura_Pista_C))

f1$Agresividad_scaled = scale(f1$Puntuacion_Agresividad_Piloto)
f1$Humedad_scaled = scale(f1$Humedad_Porcentaje)
f1$Viento_scaled = scale(f1$Velocidad_Viento_KMH)
f1$Temperatura_Pista_scaled = scale(f1$Temperatura_Pista_C)
f1$Compuesto_Neumatico = as.factor(f1$Compuesto_Neumatico)
```

### 0.3.6 Lasso (II): Motor 1 (Degradación)

Se construye la matriz de diseño y se ajusta una regresión Lasso con validación cruzada (cv.glmnet,  $\alpha = 1$ ) para la variación del tiempo de vuelta, con el objetivo de seleccionar qué predictores explican mejor el ritmo de carrera (Motor 1: Degradación).

```
# --- Lasso (II): Motor 1 (Degradacion) ---
datos_lasso_m1 = na.omit(f1[, c("Variacion_Tiempo_Vuelta",
  ↪ "Compuesto_Neumatico",
                                "Temperatura_Pista_scaled", "Humedad_scaled",
                                ↪
                                "Viento_scaled", "Agresividad_scaled")])
x_vars_m1 = model.matrix(Variacion_Tiempo_Vuelta ~ ., data =
  ↪ datos_lasso_m1)[, -1]
y_var_m1 = datos_lasso_m1$Variacion_Tiempo_Vuelta

set.seed(123)
lasso_cv_m1 = cv.glmnet(x_vars_m1, y_var_m1, alpha = 1)
print(coef(lasso_cv_m1, s = "lambda.min"))
```

```
9 x 1 sparse Matrix of class "dgCMatrix"

                                s0
(Intercept)                    0.033768725
Compuesto_NeumaticoHARD         .
Compuesto_NeumaticoINTERMEDIATE 0.004797483
Compuesto_NeumaticoSOFT         .
Compuesto_NeumaticoWET          0.004099443
Temperatura_Pista_scaled        .
Humedad_scaled                  .
Viento_scaled                   .
Agresividad_scaled              0.087670464
```

### 0.3.7 Lasso (III): Motor 2 (Vida Útil)

Se repite el mismo procedimiento de Lasso con validación cruzada, en este caso utilizando la duración del stint como variable respuesta, para seleccionar los predictores más relevantes de la vida útil del neumático (Motor 2: Vida Útil).

```
# --- Lasso (III): Motor 2 (Vida Util) ---
datos_lasso_m2 = na.omit(f1[, c("Duracion_Stint", "Compuesto_Neumatico",
  ↪ "Temperatura_Pista_scaled", "Humedad_scaled",
                                ↪
                                "Viento_scaled", "Agresividad_scaled")])
x_vars_m2 = model.matrix(Duracion_Stint ~ ., data = datos_lasso_m2)[, -1]
y_var_m2 = datos_lasso_m2$Duracion_Stint

set.seed(123)
```

```
lasso_cv_m2 = cv.glmnet(x_vars_m2, y_var_m2, alpha = 1)
print(coef(lasso_cv_m2, s = "lambda.min"))
```

```
9 x 1 sparse Matrix of class "dgCMatrix"

              s0
(Intercept)    19.14208310
Compuesto_NeumaticoHARD    7.49785209
Compuesto_NeumaticoINTERMEDIATE -5.01288483
Compuesto_NeumaticoSOFT    -1.85251589
Compuesto_NeumaticoWET    -14.18011960
Temperatura_Pista_scaled    0.07168185
Humedad_scaled    1.00367802
Viento_scaled    -0.52176415
Agresividad_scaled    -1.56055860
```

### 0.3.8 Geometría Lasso vs Ridge (I): Entorno Gráfico

Se prepara el entorno gráfico base (con tipografía serif y una rejilla de dos paneles) y se define la función `dibujar_elipses()`, que traza las curvas de nivel elípticas de la función de pérdida alrededor de un estimador de referencia, como paso previo a comparar visualmente las regiones de restricción de Lasso y Ridge.

```
# --- Geometria Lasso vs Ridge (I): Entorno Grafico ---
par(mfrow = c(1, 2), mar = c(4, 4, 2, 2), family = "serif")
color_restriccion = "#287233"
color_elipse = "#ab5c5c"
cx = 1.5; cy = 2.5; angulo_elipse = pi / 4
eje_mayor = 1.5 * sqrt(2); eje_menor = 0.75

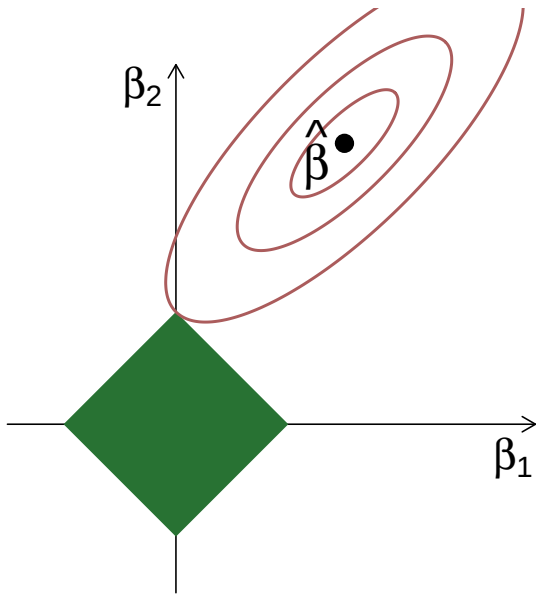
dibujar_elipses = function(cx, cy, a, b, angulo, escalas) {
  theta = seq(0, 2 * pi, length = 500)
  for (s in escalas) {
    x = a * s * cos(theta)
    y = b * s * sin(theta)
    x_rot = x * cos(angulo) - y * sin(angulo)
    y_rot = x * sin(angulo) + y * cos(angulo)
    lines(cx + x_rot, cy + y_rot, col = color_elipse, lwd = 1.5) } }
}
```

### 0.3.9 Geometría Lasso vs Ridge (II): Lasso

Se construye la representación geométrica clásica de la regularización Lasso: se dibuja la región de restricción en forma de rombo (norma L1) junto con las elipses de nivel de la función de pérdida, ilustrando por qué la intersección suele producirse en un vértice y, por tanto, por qué Lasso anula coeficientes.

```
# --- Geometria Lasso vs Ridge (II): Lasso ---
configurar_ejes = function() {
  plot(0, 0, type = "n", xlim = c(-1.5, 3.5), ylim = c(-1.5, 3.5), axes =
    ↪ FALSE, xlab = "", ylab = "", asp = 1)
  arrows(-1.5, 0, 3.2, 0, length = 0.08, lwd = 0.8)
  arrows(0, -1.5, 0, 3.2, length = 0.08, lwd = 0.8)
  text(3.0, -0.3, expression(beta[1]), cex = 1.3)
  text(-0.3, 3.0, expression(beta[2]), cex = 1.3) }

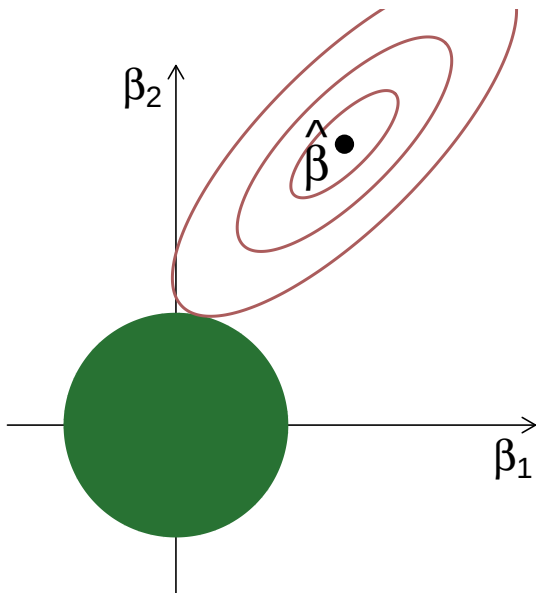
configurar_ejes()
polygon(x = c(-1, 0, 1, 0), y = c(0, 1, 0, -1), col = color_restriccion,
  ↪ border = NA)
escalas_lasso = c(0.3, 0.6, 1.0)
dibujar_elipses(cx, cy, eje_mayor, eje_menor, angulo_elipse, escalas_lasso)
points(cx, cy, pch = 19, col = "black", cex = 1.2)
text(cx - 0.25, cy - 0.1, expression(hat(beta)), cex = 1.5)
```



### 0.3.10 Geometría Lasso vs Ridge (III): Ridge

Se construye, de forma análoga, la representación geométrica de la regularización Ridge: la región de restricción circular (norma L2) y las mismas elipses de nivel, calculando además la escala exacta a la que la elipse es tangente al círculo, lo que ilustra por qué Ridge encoge los coeficientes sin anularlos.

```
# --- Geometria Lasso vs Ridge (III): Ridge ---
configurar_ejes()
theta_circulo = seq(0, 2 * pi, length = 500)
polygon(x = cos(theta_circulo), y = sin(theta_circulo), col =
  ↪ color_restriccion, border = NA)
t_c = seq(0, 2 * pi, length.out = 5000)
x_c = cos(t_c)
y_c = sin(t_c)
u_c = (x_c - cx) * cos(-angulo_elipse) - (y_c - cy) * sin(-angulo_elipse)
v_c = (x_c - cx) * sin(-angulo_elipse) + (y_c - cy) * cos(-angulo_elipse)
escala_tangente_ridge = min(sqrt((u_c / eje_mayor)^2 + (v_c / eje_menor)^2))
escalas_ridge = c(0.3, 0.6, escala_tangente_ridge)
dibujar_elipses(cx, cy, eje_mayor, eje_menor, angulo_elipse, escalas_ridge)
points(cx, cy, pch = 19, col = "black", cex = 1.2)
text(cx - 0.25, cy - 0.1, expression(hat(beta)), cex = 1.5)
```



## 0.4 Modelización: Modelos Lineales Mixtos (LMM)

### 0.4.1 Estimación Base OLS: Motor 1

Se ajusta un modelo de regresión lineal simple (OLS) de la variación del tiempo de vuelta en función de la agresividad estandarizada del piloto, que servirá como modelo base de comparación frente al modelo mixto del Motor 1.

```
# --- Estimacion Base OLS: Motor 1 ---
modelo_base_m1 = lm(Variacion_Tiempo_Vuelta ~ Agresividad_scaled, data = f1)
print(summary(modelo_base_m1)$coefficients)
```

|                    | Estimate   | Std. Error   | t value   | Pr(> t ) |
|--------------------|------------|--------------|-----------|----------|
| (Intercept)        | 0.03407498 | 7.631478e-05 | 446.5056  | 0        |
| Agresividad_scaled | 0.08818267 | 7.632009e-05 | 1155.4319 | 0        |

### 0.4.2 Estimación Base OLS: Motor 2

Se ajusta un modelo OLS de la duración del stint en función del compuesto de neumático, la agresividad, la humedad, el viento y la temperatura de pista, que actuará como modelo base de comparación para el Motor 2.

```
# --- Estimacion Base OLS: Motor 2 ---
f1$Temp_Pista_final = as.numeric(f1$Temperatura_Pista_scaled)

modelo_base_m2 = lm(Duracion_Stint ~ Compuesto_Neumatico + Agresividad_scaled
  ↪ +
  ↪ Humedad_scaled + Viento_scaled + Temp_Pista_final, data
  ↪ = f1)
print(summary(modelo_base_m2)$coefficients)
```

|                                 | Estimate     | Std. Error | t value     |
|---------------------------------|--------------|------------|-------------|
| (Intercept)                     | 19.15314041  | 0.2370617  | 80.7939170  |
| Compuesto_NeumaticoHARD         | 7.50509509   | 0.3493687  | 21.4818735  |
| Compuesto_NeumaticoINTERMEDIATE | -5.07208304  | 0.6124084  | -8.2821902  |
| Compuesto_NeumaticoSOFT         | -1.88039235  | 0.3572029  | -5.2642133  |
| Compuesto_NeumaticoWET          | -14.32232056 | 1.4118393  | -10.1444411 |
| Agresividad_scaled              | -1.57467018  | 0.1396186  | -11.2783659 |
| Humedad_scaled                  | 1.02442748   | 0.1477982  | 6.9312570   |
| Viento_scaled                   | -0.53442505  | 0.1416745  | -3.7722024  |
| Temp_Pista_final                | 0.08322527   | 0.1462431  | 0.5690886   |

|                                 | Pr(> t )     |
|---------------------------------|--------------|
| (Intercept)                     | 0.000000e+00 |
| Compuesto_NeumaticoHARD         | 4.630692e-99 |
| Compuesto_NeumaticoINTERMEDIATE | 1.446103e-16 |
| Compuesto_NeumaticoSOFT         | 1.451522e-07 |
| Compuesto_NeumaticoWET          | 5.219444e-24 |
| Agresividad_scaled              | 3.068954e-29 |
| Humedad_scaled                  | 4.559197e-12 |
| Viento_scaled                   | 1.632027e-04 |
| Temp_Pista_final                | 5.693150e-01 |

### 0.4.3 Entrenamiento LMM: Motor 1 (Degradación)

Se entrena el modelo lineal mixto definitivo del Motor 1: la variación del tiempo de vuelta en función de la agresividad del piloto, con intercepto aleatorio por circuito, piloto y constructor, estimado por máxima verosimilitud (REML = FALSE). Se muestran la varianza de cada efecto aleatorio y los coeficientes fijos estimados.

```
# --- Entrenamiento LMM: Motor 1 (Degradacion) ---
library(lme4)
modelo_lmer_m1 = lmer(Variacion_Tiempo_Vuelta ~ Agresividad_scaled +
                      (1 | Circuito) + (1 | Piloto) + (1 | Constructor),
                      data = f1, REML = FALSE)
print(VarCorr(modelo_lmer_m1), comp = "Variance")
```

| Groups      | Name        | Variance   |
|-------------|-------------|------------|
| Piloto      | (Intercept) | 3.4713e-07 |
| Circuito    | (Intercept) | 1.2480e-05 |
| Constructor | (Intercept) | 6.9202e-07 |
| Residual    |             | 3.2445e-05 |

```
print(fixef(modelo_lmer_m1))
```

|             |                    |
|-------------|--------------------|
| (Intercept) | Agresividad_scaled |
| 0.03417747  | 0.08765658         |

### 0.4.4 Entrenamiento LMM: Motor 2 (Vida Útil)

Se entrena el modelo lineal mixto definitivo del Motor 2: la duración del stint en función del compuesto de neumático, la agresividad, la humedad, el viento y la temperatura de pista, con los mismos tres efectos aleatorios jerárquicos que el Motor 1.



```
# --- Entrenamiento LMM: Motor 2 (Vida Util) ---
library(lme4)
modelo_lmer_m2 = lmer(Duracion_Stint ~ Compuesto_Neumatico +
  ↪ Agresividad_scaled +
                        Humedad_scaled + Viento_scaled + Temp_Pista_final +
                        (1 | Circuito) + (1 | Piloto) + (1 | Constructor),
                        data = f1, REML = FALSE)
print(VarCorr(modelo_lmer_m2), comp = "Variance")
```

| Groups      | Name        | Variance |
|-------------|-------------|----------|
| Piloto      | (Intercept) | 2.1535   |
| Circuito    | (Intercept) | 24.1380  |
| Constructor | (Intercept) | 5.5830   |
| Residual    |             | 113.4619 |

```
print(fixef(modelo_lmer_m2))
```

|                                 |                         |
|---------------------------------|-------------------------|
| (Intercept)                     | Compuesto_NeumaticoHARD |
| 20.3996486                      | 7.6294185               |
| Compuesto_NeumaticoINTERMEDIATE | Compuesto_NeumaticoSOFT |
| -8.5136192                      | -2.9207172              |
| Compuesto_NeumaticoWET          | Agresividad_scaled      |
| -17.2677065                     | -1.2950994              |
| Humedad_scaled                  | Viento_scaled           |
| -0.5919284                      | 0.4270672               |
| Temp_Pista_final                |                         |
| -0.6751896                      |                         |

#### 0.4.5 Efectos Aleatorios (Lollipop Plot)

Se extraen los interceptos aleatorios estimados por circuito del Motor 2 y se representan en un gráfico de tipo “piruleta” (lollipop), ordenado de menor a mayor efecto y coloreado en verde o rojo según si el circuito favorece o penaliza la vida útil del neumático.

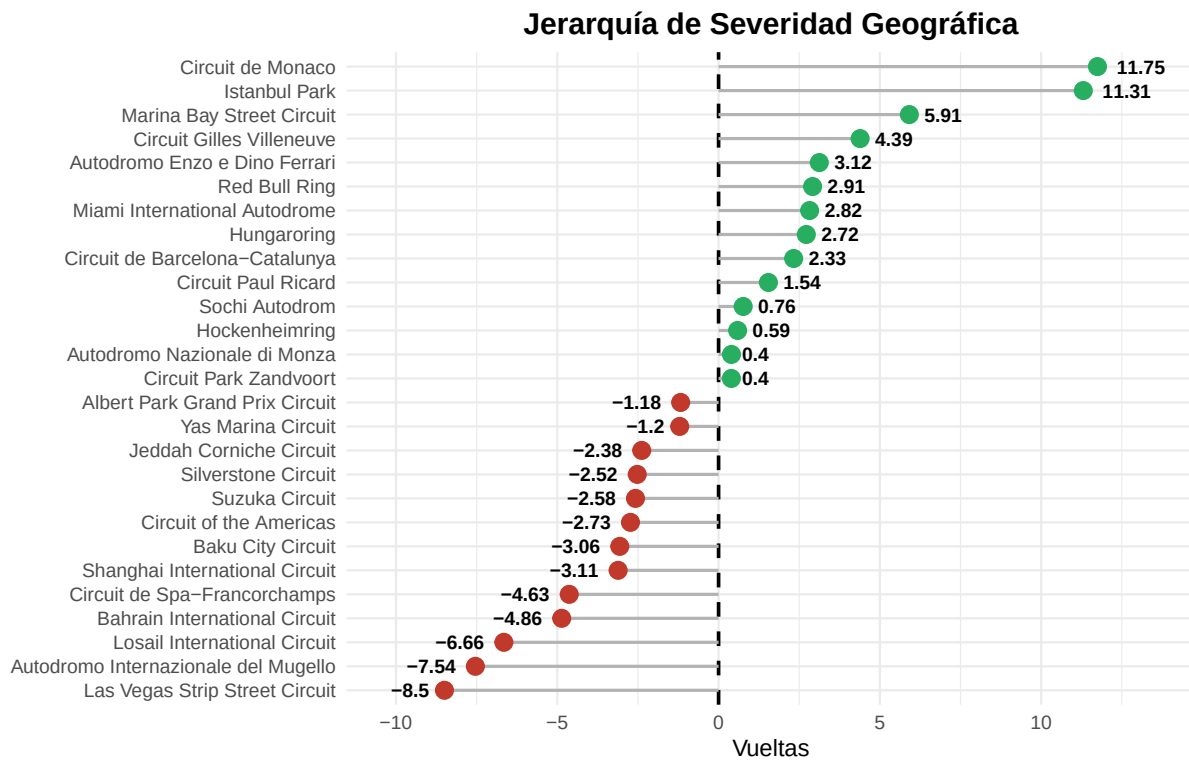
```
# --- Efectos Aleatorios (Lollipop Plot) ---
library(ggplot2)
efectos_re = ranef(modelo_lmer_m2)$Circuito
df_circuitos = data.frame(Circuito = rownames(efectos_re), Efecto =
  ↪ efectos_re$`(Intercept)`)
df_circuitos$Circuito = reorder(df_circuitos$Circuito, df_circuitos$Efecto)
```

```

plot_lollipop = ggplot(df_circuitos, aes(x = Circuito, y = Efecto)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black", linewidth
    ↪ = 0.8) +
  geom_segment(aes(x = Circuito, xend = Circuito, y = 0, yend = Efecto),
    color = "gray70", linewidth = 0.7) +
  geom_point(aes(color = Efecto > 0), size = 3.5, show.legend = FALSE) +
  scale_color_manual(values = c("TRUE" = "#27AE60", "FALSE" = "#C0392B")) +
  geom_text(aes(label = round(Efecto, 2)),
    hjust = ifelse(df_circuitos$Efecto > 0, -0.4, 1.4), size = 3,
    ↪ fontface = "bold") +
  scale_y_continuous(expand = expansion(mult = c(0.15, 0.15))) +
  coord_flip(clip = "off") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 14)) +
  labs(title = "Jerarquía de Severidad Geográfica", x = "", y = "Vueltas")

print(plot_lollipop)

```



### 0.4.6 Efectos Fijos (Forest Plot)

Se extraen los coeficientes fijos del Motor 2 junto con sus errores estándar, se traducen sus nombres técnicos a etiquetas legibles, se calculan los intervalos de confianza al 95% y se representan en un forest plot que resume el impacto neto de cada predictor.

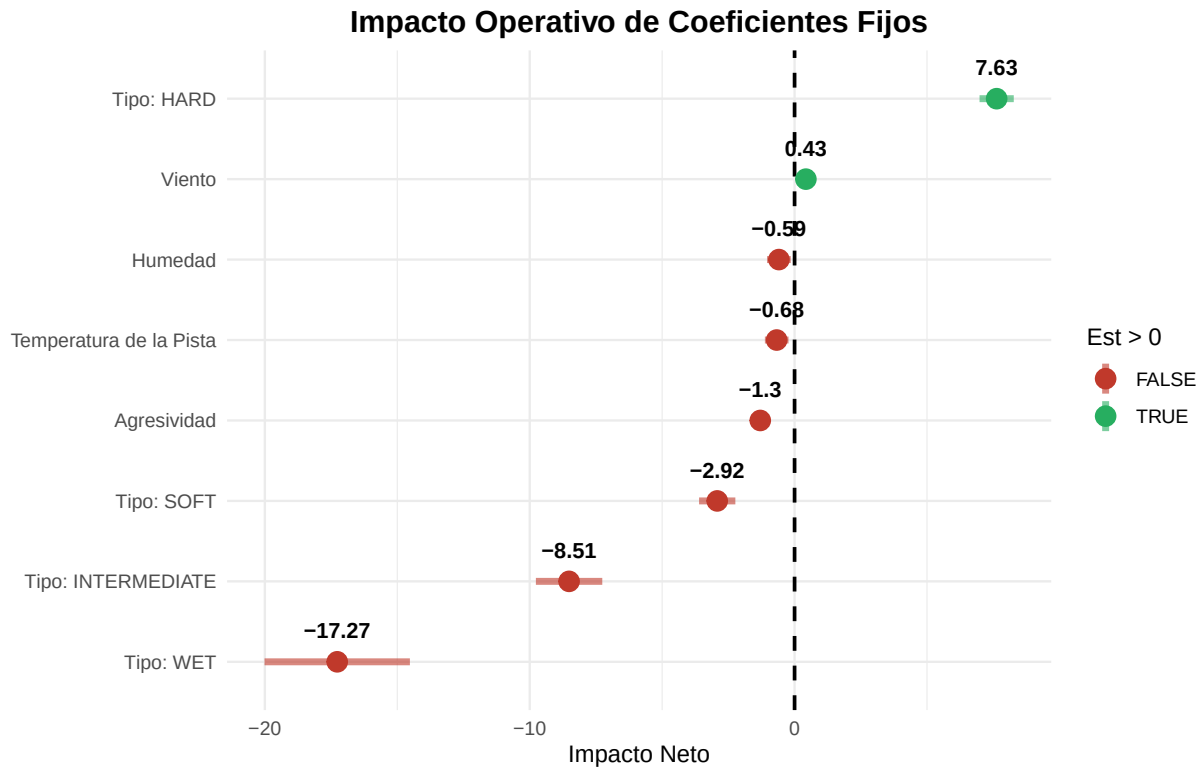
```
# --- Efectos Fijos (Forest Plot) ---
resumen = as.data.frame(coef(summary(modelo_lmer_m2)))[-1, ]
df_fijos = data.frame(Var = rownames(resumen), Est = resumen$Estimate, SE =
  ↪ resumen$`Std. Error`)

df_fijos$Var = gsub("Compuesto_Neumatico", "Tipo: ", df_fijos$Var)
df_fijos$Var = gsub("Agresividad_scaled", "Agresividad", df_fijos$Var)
df_fijos$Var = gsub("Humedad_scaled", "Humedad", df_fijos$Var)
df_fijos$Var = gsub("Viento_scaled", "Viento", df_fijos$Var)
df_fijos$Var = gsub("Temp_Pista_final", "Temperatura de la Pista",
  ↪ df_fijos$Var)

df_fijos$min = df_fijos$Est - (1.96 * df_fijos$SE)
df_fijos$max = df_fijos$Est + (1.96 * df_fijos$SE)
df_fijos$Var = reorder(df_fijos$Var, df_fijos$Est)

plot_fijos = ggplot(df_fijos, aes(x = Var, y = Est, color = Est > 0)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "black", linewidth
  ↪ = 0.8) +
  geom_linerange(aes(ymin = min, ymax = max), linewidth = 1.5, alpha = 0.6) +
  geom_point(size = 4) +
  scale_color_manual(values = c("TRUE" = "#27AE60", "FALSE" = "#C0392B")) +
  geom_text(aes(label = round(Est, 2)), vjust = -1.5, size = 3.5, fontface =
  ↪ "bold", color = "black") +
  coord_flip(clip = "off") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 14)) +
  labs(title = "Impacto Operativo de Coeficientes Fijos", x = "", y =
  ↪ "Impacto Neto")

print(plot_fijos)
```



#### 0.4.7 Test de Razón de Verosimilitud (LRT): Motor 1

Se aplica un test de razón de verosimilitud (LRT), mediante `anova()`, entre el modelo mixto y el modelo OLS base del Motor 1, para contrastar si la inclusión de los efectos aleatorios jerárquicos mejora de forma significativa el ajuste.

```
# --- Test de Razon de Verosimilitud (LRT): Motor 1 ---
print(anova(modelo_lmer_m1, modelo_base_m1))
```

Data: f1

Models:

modelo\_base\_m1: Variacion\_Tiempo\_Vuelta ~ Agresividad\_scaled

modelo\_lmer\_m1: Variacion\_Tiempo\_Vuelta ~ Agresividad\_scaled + (1 | Circuito)  
 ↪ + (1 | Piloto) + (1 | Constructor)

|                | npar | AIC    | BIC    | logLik | deviance | Chisq  | Df | Pr(>Chisq)    |
|----------------|------|--------|--------|--------|----------|--------|----|---------------|
| modelo_base_m1 | 3    | -52067 | -52046 | 26036  | -52073   |        |    |               |
| modelo_lmer_m1 | 6    | -53697 | -53655 | 26854  | -53709   | 1636.1 | 3  | < 2.2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

#### 0.4.8 Test de Razón de Verosimilitud (LRT): Motor 2

Se realiza el mismo contraste de razón de verosimilitud (LRT) entre el modelo mixto y el modelo OLS base, pero para el Motor 2.

```
# --- Test de Razon de Verosimilitud (LRT): Motor 2 ---  
print(anova(modelo_lmer_m2, modelo_base_m2))
```

Data: f1

Models:

modelo\_base\_m2: Duracion\_Stint ~ Compuesto\_Neumatico + Agresividad\_scaled +  
↪ Humedad\_scaled + Viento\_scaled + Temp\_Pista\_final

modelo\_lmer\_m2: Duracion\_Stint ~ Compuesto\_Neumatico + Agresividad\_scaled +  
↪ Humedad\_scaled + Viento\_scaled + Temp\_Pista\_final + (1 | Circuito) + (1 |  
↪ Piloto) + (1 | Constructor)

|                | npar | AIC   | BIC   | logLik | deviance | Chisq  | Df | Pr(>Chisq)    |
|----------------|------|-------|-------|--------|----------|--------|----|---------------|
| modelo_base_m2 | 10   | 51960 | 52028 | -25970 | 51940    |        |    |               |
| modelo_lmer_m2 | 13   | 51169 | 51258 | -25572 | 51143    | 796.82 | 3  | < 2.2e-16 *** |

---

Signif. codes: 0 '\*\*\*' 0.001 '\*\*' 0.01 '\*' 0.05 '.' 0.1 ' ' 1

#### 0.4.9 Validación Estructural AIC/BIC: Tabulación de Métricas

Se reajustan ambos pares de modelos (OLS y LMM) restringidos exactamente al mismo subconjunto de filas sin valores nulos —para garantizar una comparación ceteris paribus— y se tabulan los grados de libertad, el AIC y el BIC de cada modelo, junto con sus diferencias ( $\Delta$ AIC,  $\Delta$ BIC) respecto al modelo base.

```
# --- Validacion Estructural AIC/BIC: Tabulacion de Metricas ---  
library(lme4)  
  
# Reajuste exento de valores nulos para garantizar el principio Ceteris  
↪ Paribus  
datos_aic_m1 = na.omit(f1[, c("Variacion_Tiempo_Vuelta",  
↪ "Agresividad_scaled",  
↪ "Circuito", "Piloto", "Constructor")])  
base_lm_m1_c = lm(Variacion_Tiempo_Vuelta ~ Agresividad_scaled, data =  
↪ datos_aic_m1)  
mixto_lmer_m1_c = lmer(Variacion_Tiempo_Vuelta ~ Agresividad_scaled +  
↪ (1 | Circuito) + (1 | Piloto) + (1 | Constructor),  
↪ data = datos_aic_m1, REML = FALSE)
```

```

datos_aic_m2 = na.omit(f1[, c("Duracion_Stint", "Compuesto_Neumatico",
  ↪ "Humedad_scaled",
                                "Viento_scaled", "Agresividad_scaled",
                                ↪ "Temp_Pista_final",
                                "Circuito", "Piloto", "Constructor")])
base_lm_m2_c = lm(Duracion_Stint ~ Compuesto_Neumatico + Humedad_scaled +
  ↪ Viento_scaled +
                                Agresividad_scaled + Temp_Pista_final, data =
  ↪ datos_aic_m2)
mixto_lmer_m2_c = lmer(Duracion_Stint ~ Compuesto_Neumatico + Humedad_scaled
  ↪ + Viento_scaled +
                                Agresividad_scaled + Temp_Pista_final +
                                (1 | Circuito) + (1 | Piloto) + (1 | Constructor),
                                data = datos_aic_m2, REML = FALSE)

extraer_metricas = function(modelo_base, modelo_mixto, nombre_motor) {
  df = data.frame(
    Motor = nombre_motor, Modelo = c("OLS", "LMM"),
    Grados_Libertad = c(attr(logLik(modelo_base), "df"),
  ↪ attr(logLik(modelo_mixto), "df")),
    AIC = c(AIC(modelo_base), AIC(modelo_mixto)),
    BIC = c(BIC(modelo_base), BIC(modelo_mixto))
  )
  df$Delta_AIC = c("-", round(df$AIC[2] - df$AIC[1], 2))
  df$Delta_BIC = c("-", round(df$BIC[2] - df$BIC[1], 2))
  return(df) }

tabla_criterios = rbind(extraer_metricas(base_lm_m1_c, mixto_lmer_m1_c, "1"),
  ↪
                                extraer_metricas(base_lm_m2_c, mixto_lmer_m2_c, "2"))
print(tabla_criterios, row.names = FALSE)

```

| Motor | Modelo | Grados_Libertad | AIC       | BIC       | Delta_AIC | Delta_BIC |
|-------|--------|-----------------|-----------|-----------|-----------|-----------|
| 1     | OLS    | 3               | -52066.67 | -52046.03 | -         | -         |
| 1     | LMM    | 6               | -53696.73 | -53655.45 | -1630.06  | -1609.42  |
| 2     | OLS    | 10              | 51959.91  | 52028.06  | -         | -         |
| 2     | LMM    | 13              | 51169.09  | 51257.68  | -790.82   | -770.37   |

#### 0.4.10 Corrección ICC: Singularidad Escalar en Motor 1

Se reescala la variable respuesta del Motor 1 a milisegundos para evitar el problema numérico de “singularidad escalar” al calcular el Coeficiente de Correlación Intraclass (ICC) por grupo, y se calcula también el ICC del Motor 2.

```
# --- Correccion ICC: Singularidad Escalar en Motor 1 ---
library(performance)

modelo_icc_m1_milisegundos = lmer(I(Variacion_Tiempo_Vuelta * 1000) ~
  ↪ Agresividad_scaled +
                                     (1 | Circuito) + (1 | Piloto) + (1 |
  ↪ Constructor),
                                     data = f1, REML = FALSE)

print(icc(modelo_icc_m1_milisegundos, by_group = TRUE))
```

```
# ICC by Group
```

| Group       |  | ICC   |
|-------------|--|-------|
| Piloto      |  | 0.008 |
| Circuito    |  | 0.272 |
| Constructor |  | 0.015 |

```
print(icc(modelo_lmer_m2, by_group = TRUE))
```

```
# ICC by Group
```

| Group       |  | ICC   |
|-------------|--|-------|
| Piloto      |  | 0.015 |
| Circuito    |  | 0.166 |
| Constructor |  | 0.038 |

#### 0.4.11 Extracción de Pseudo R-Cuadrado de Nakagawa

Se extrae el pseudo  $R^2$  de Nakagawa (marginal y condicional) de ambos modelos mixtos, lo que permite cuantificar, respectivamente, la varianza explicada solo por los efectos fijos y por el modelo completo (fijos + aleatorios).

```
# --- Extraccion de Pseudo R-Cuadrado de Nakagawa ---
print(r2(modelo_icc_m1_milisegundos))
```

```
# R2 for Mixed Models
```

```
Conditional R2: 0.996
Marginal R2: 0.994
```

```
print(r2(modelo_lmer_m2))
```

```
# R2 for Mixed Models
```

```
Conditional R2: 0.350
Marginal R2: 0.167
```

#### 0.4.12 Gráfico de Varianza Explicada (ICC)

A partir de los valores de ICC calculados, se construye un gráfico de barras agrupadas que muestra el porcentaje de varianza explicada por Circuito, Constructor y Piloto en cada uno de los dos motores del modelo.

```
# --- Grafico de Varianza Explicada (ICC) ---
library(ggplot2)

df_icc = data.frame(
  Motor = c(rep("Motor 1 (Degradación)", 3), rep("Motor 2 (Vida Útil)", 3)),
  Agente = c("Circuito", "Constructor", "Piloto", "Circuito", "Constructor",
    ↪ "Piloto"),
  Porcentaje = c(27.2, 1.5, 0.8, 16.6, 3.8, 1.5))
df_icc$Agente = factor(df_icc$Agente, levels = c("Circuito", "Constructor",
  ↪ "Piloto"))

grafico_icc = ggplot(df_icc, aes(x = Motor, y = Porcentaje, fill = Agente)) +
  geom_bar(stat = "identity", position = position_dodge(width = 0.7), width =
    ↪ 0.6, color = "black") +
  geom_text(aes(label = paste0(Porcentaje, "%")), position =
    ↪ position_dodge(width = 0.7), vjust = -0.8, size = 3.5, fontface =
    ↪ "bold") +
  scale_fill_manual(values = c("Circuito" = "#2C3E50", "Constructor" =
    ↪ "#7F8C8D", "Piloto" = "#2980B9")) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.15))) +
```

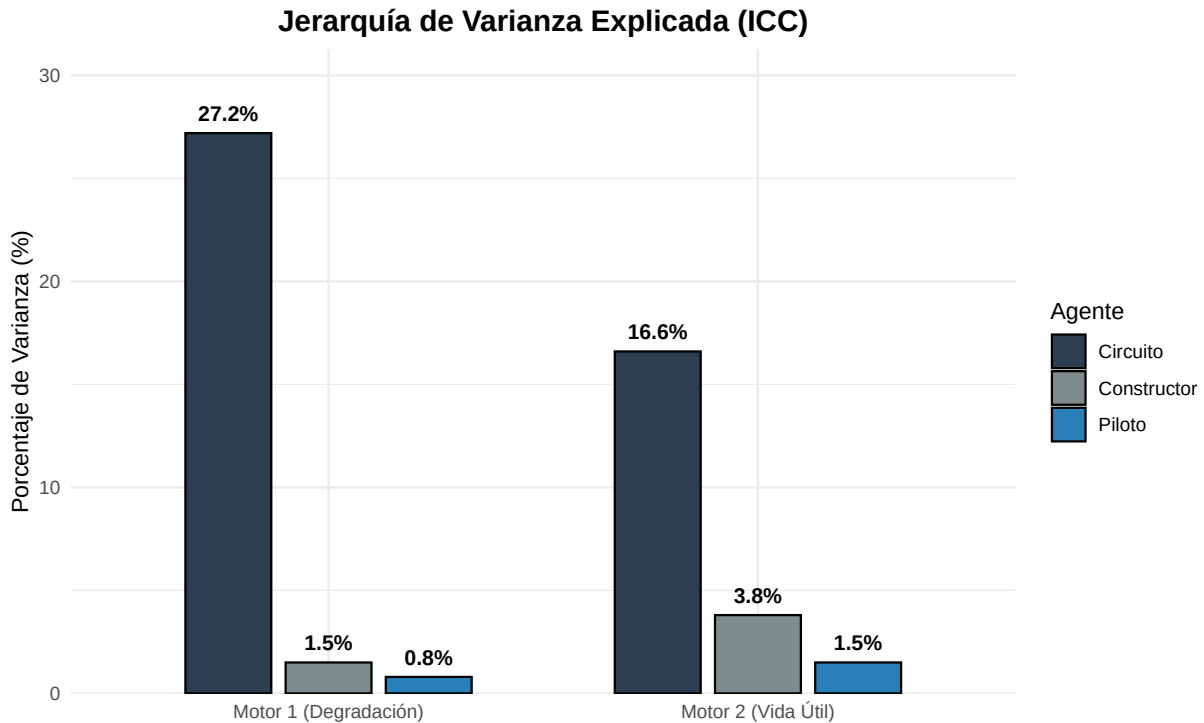


```

theme_minimal() +
theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 14)) +
labs(title = "Jerarquía de Varianza Explicada (ICC)", x = "", y =
  ↪ "Porcentaje de Varianza (%)")

print(grafico_icc)

```



#### 0.4.13 Validación de Supuestos Matemáticos

Se generan los gráficos de diagnóstico habituales del Motor 2 —residuos frente a valores ajustados, para evaluar la homocedasticidad, y un gráfico Q-Q, para evaluar la normalidad de los residuos— y se combinan en un único panel mediante la librería patchwork.

```

# --- Validacion de Supuestos Matematicos ---
library(ggplot2)
library(patchwork)

df_residuos = data.frame(Ajustados = fitted(modelo_lmer_m2), Residuos =
  ↪ resid(modelo_lmer_m2))

```

```

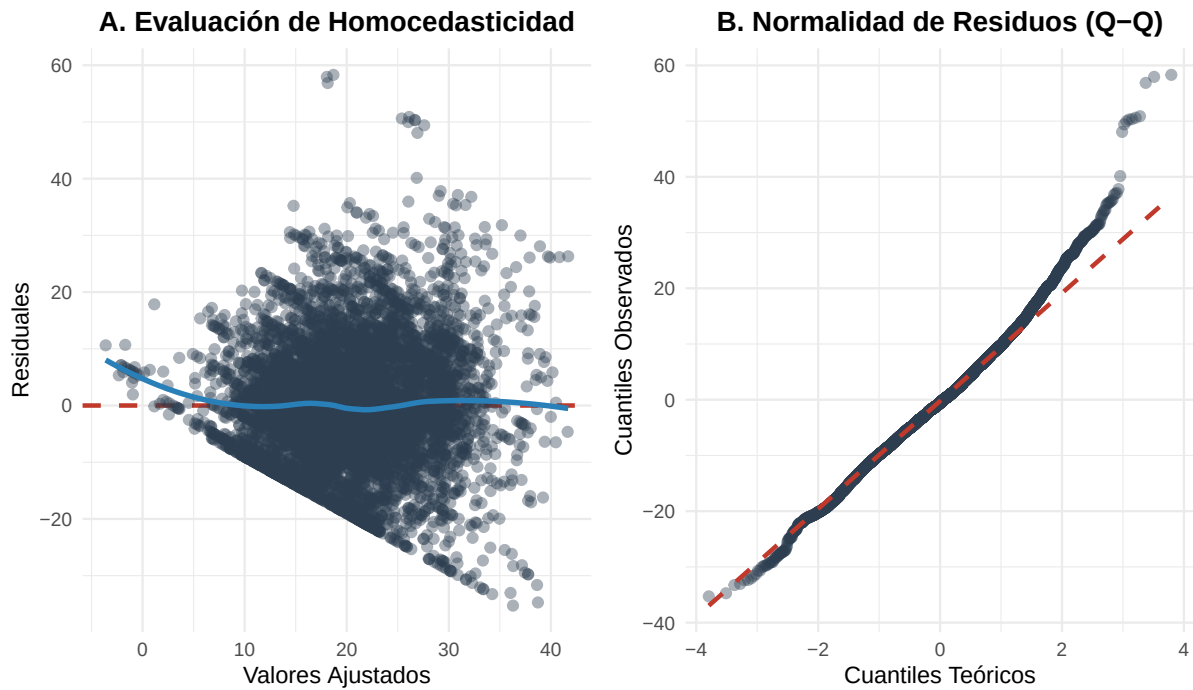
plot_homocedasticidad = ggplot(df_residuos, aes(x = Ajustados, y = Residuos))
  ↪ +
  geom_hline(yintercept = 0, linetype = "dashed", color = "#C0392B",
    ↪ linewidth = 1) +
  geom_point(alpha = 0.4, color = "#2C3E50", size = 2) +
  geom_smooth(method = "loess", se = FALSE, color = "#2980B9", linewidth =
    ↪ 1.2) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold")) +
  labs(title = "A. Evaluación de Homocedasticidad", x = "Valores Ajustados",
    ↪ y = "Residuales")

plot_qq = ggplot(df_residuos, aes(sample = Residuos)) +
  stat_qq(alpha = 0.4, color = "#2C3E50", size = 2) +
  stat_qq_line(color = "#C0392B", linewidth = 1, linetype = "dashed") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold")) +
  labs(title = "B. Normalidad de Residuos (Q-Q)", x = "Cuantiles Teóricos", y
    ↪ = "Cuantiles Observados")

grafica_diagnostico = plot_homocedasticidad + plot_qq +
  plot_annotation(title = 'Validación de Supuestos Matemáticos',
    theme = theme(plot.title = element_text(hjust = 0.5, face =
      ↪ "bold", size = 16)))
print(grafica_diagnostico)

```

## Validación de Supuestos Matemáticos



### 0.4.14 Predicciones Marginales Termodinámicas

Se utiliza la librería `ggeffects` para predecir la duración del stint en función de la agresividad del piloto, desagregando por compuesto de neumático, y se representan las curvas predichas junto con sus bandas de confianza, ordenadas según la dureza habitual de cada compuesto.

```
# --- Predicciones Marginales Termodinamicas ---
library(ggeffects)
library(ggplot2)

predicciones = ggpredict(modelo_lmer_m2, terms = c("Agresividad_scaled
  ↪ [all]", "Compuesto_Neumatico"))
df_pred = as.data.frame(predicciones)

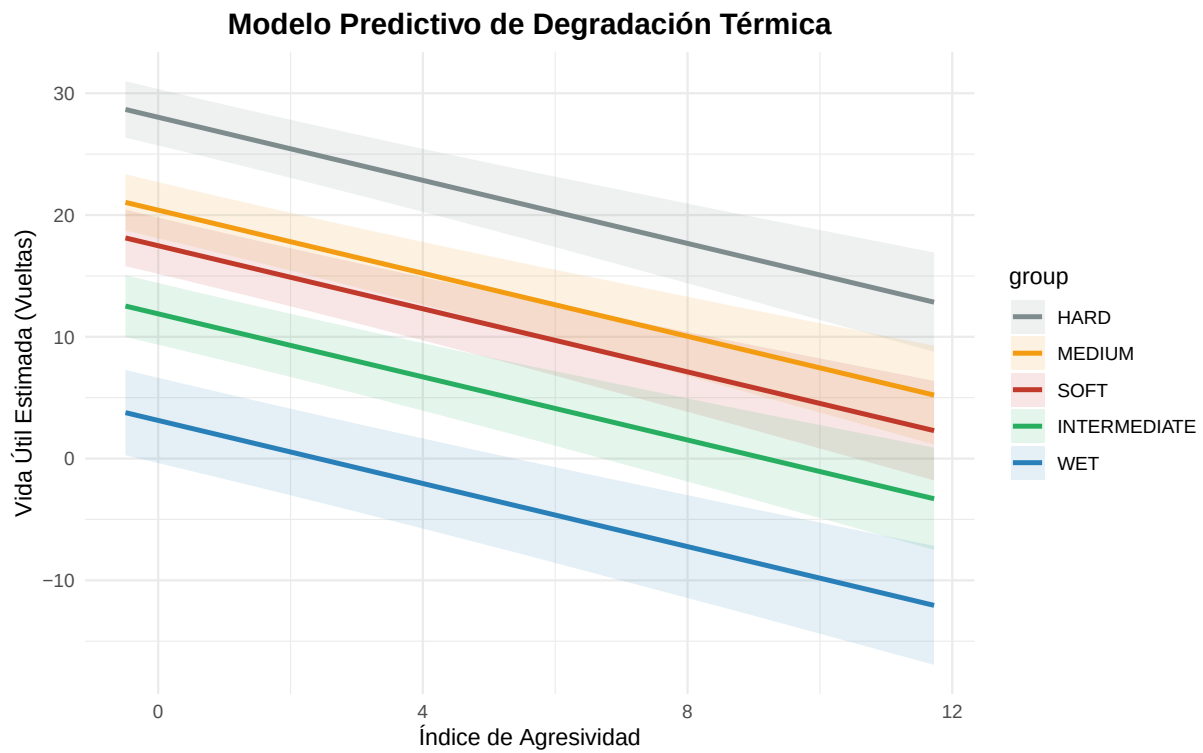
orden_dureza = c("HARD", "MEDIUM", "SOFT", "INTERMEDIATE", "WET")
df_pred$group = factor(df_pred$group, levels = intersect(orden_dureza,
  ↪ unique(df_pred$group)))
colores_f1 = c("HARD" = "#7F8C8D", "MEDIUM" = "#F39C12", "SOFT" = "#C0392B",
  ↪ "INTERMEDIATE" = "#27AE60", "WET" = "#2980B9")
```

```

plot_marginales = ggplot(df_pred, aes(x = x, y = predicted, color = group,
  ↪ fill = group)) +
  geom_ribbon(aes(ymin = conf.low, ymax = conf.high), alpha = 0.12, color =
    ↪ NA) +
  geom_line(linewidth = 1.1) +
  scale_color_manual(values = colores_f1) +
  scale_fill_manual(values = colores_f1) +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 14)) +
  labs(title = "Modelo Predictivo de Degradación Térmica",
    x = "Índice de Agresividad", y = "Vida Útil Estimada (Vueltas)")

print(plot_marginales)

```



## 0.5 Simulación de Estrategia de Carrera (Monte Carlo)

### 0.5.1 Simulación (I): Carga y Entrenamiento Operativo

Se recarga el dataset limpio guardado anteriormente en `f1_limpiar.rds`, se reconstruyen de forma defensiva los tipos de las variables necesarias (para evitar errores o fugas de información) y se reentrenan los dos modelos mixtos (Motor 1 y Motor 2) que alimentarán el simulador de estrategia de carrera.

```
# --- Simulación (I): Carga y Entrenamiento Operativo ---
library(lme4)
library(dplyr)
library(ggplot2)

f1 = readRDS("f1_limpiar.rds")

# Inyección de seguridad para evitar errores de tipo en el LMM (Data Leakage
  ↪ Fix)
f1$Variacion_Tiempo_Vuelta =
  ↪ as.numeric(as.character(f1$Variacion_Tiempo_Vuelta))
f1$Duracion_Stint = as.numeric(as.character(f1$Duracion_Stint))

f1$Puntuacion_Agresividad_Piloto =
  ↪ as.numeric(as.character(f1$Puntuacion_Agresividad_Piloto))
f1$Agresividad_scaled = scale(f1$Puntuacion_Agresividad_Piloto)
f1$Compuesto_Neumatico = as.factor(f1$Compuesto_Neumatico)
f1$Piloto = as.factor(f1$Piloto)
f1$Circuito = as.factor(f1$Circuito)
f1$Constructor = as.factor(f1$Constructor)
f1$Temp_Pista_final = as.numeric(f1$Temperatura_Pista_scaled)

modelo_lmer_m1 = lmer(Variacion_Tiempo_Vuelta ~ Agresividad_scaled +
  (1 | Circuito) + (1 | Piloto) + (1 | Constructor),
  data = f1, REML = FALSE)

modelo_lmer_m2 = lmer(Duracion_Stint ~ Compuesto_Neumatico +
  ↪ Agresividad_scaled +
  Humedad_scaled + Viento_scaled + Temp_Pista_final +
  (1 | Circuito) + (1 | Piloto) + (1 | Constructor),
  data = f1, REML = FALSE)
```

### 0.5.2 Simulación (II): Diccionario Topográfico y Logístico

Se excluyen los circuitos que ya no forman parte del calendario actual y se construyen dos tablas resumen: una por circuito, con el número de vueltas totales, el tiempo medio de parada, y las medias de temperatura, humedad y viento; y otra por escudería, con el tiempo de parada mediano de cada equipo y su “bonus” relativo respecto a la mediana global de todas las escuderías.

```
# --- Simulacion (II): Diccionario Topografico y Logistico ---
library(dplyr)

circuitos_inactivos = c("Autódromo Internacional do Algarve", "Autodromo
↪ Internazionale del Mugello",
                        "Circuit Paul Ricard", "Hockenheimring", "Istanbul
↪ Park",
                        "Nürburgring", "Sochi Autodrom")

info_circuitos = f1 %>%
  filter(!Circuito %in% circuitos_inactivos) %>%
  mutate(
    Vueltas = as.numeric(as.character(Vueltas)),
    Tiempo_Medio_Parada_Pit =
      ↪ as.numeric(as.character(Tiempo_Medio_Parada_Pit)),
    Temperatura_Pista_C = as.numeric(as.character(Temperatura_Pista_C)),
    Humedad_Porcentaje = as.numeric(as.character(Humedad_Porcentaje)),
    Velocidad_Viento_KMH = as.numeric(as.character(Velocidad_Viento_KMH))
  ) %>%
  group_by(Circuito) %>%
  summarise(
    Vueltas_Totales = max(Vueltas, na.rm = TRUE),
    Tiempo_PitStop_Medio =
      ↪ median(Tiempo_Medio_Parada_Pit[Tiempo_Medio_Parada_Pit < 40], na.rm =
      ↪ TRUE),
    Temp_Pista_Media = mean(Temperatura_Pista_C, na.rm = TRUE),
    Humedad_Media = mean(Humedad_Porcentaje, na.rm = TRUE),
    Viento_Medio = mean(Velocidad_Viento_KMH, na.rm = TRUE),
    Temp_Pista_scaled_Media = mean(Temp_Pista_final, na.rm = TRUE),
    Humedad_scaled_Media = mean(Humedad_scaled, na.rm = TRUE),
    Viento_scaled_Media = mean(Viento_scaled, na.rm = TRUE)
  ) %>%
  mutate(
    Temp_Pista_scaled_Media = ifelse(is.na(Temp_Pista_scaled_Media), 0,
      ↪ Temp_Pista_scaled_Media),
```

```

    Humedad_scaled_Media = ifelse(is.na(Humedad_scaled_Media), 0,
      ↪ Humedad_scaled_Media),
    Viento_scaled_Media = ifelse(is.na(Viento_scaled_Media), 0,
      ↪ Viento_scaled_Media)
  ) %>% ungroup() %>% as.data.frame()

f1$Tiempo_Pit_Num = as.numeric(as.character(f1$Tiempo_Medio_Parada_Pit))
mediana_global_pit = median(f1$Tiempo_Pit_Num[f1$Tiempo_Pit_Num < 40], na.rm
  ↪ = TRUE)

info_escuderias = f1 %>%
  filter(Tiempo_Pit_Num < 40) %>%
  group_by(Constructor) %>%
  summarise(
    PitStop_Mediano_Escuderia = median(Tiempo_Pit_Num, na.rm = TRUE),
    Bonus_Mecanicos = PitStop_Mediano_Escuderia - mediana_global_pit
  ) %>% as.data.frame()

```

### 0.5.3 Simulación (III): Consola del Muro

Se define la función `preparar_muro()`, que construye la lista de parámetros de una carrera concreta a simular —circuitos, piloto, constructor, posición de salida y si se fuerza lluvia— calculando además la penalización por tráfico y la agresividad adicional asociadas a la posición de salida.

```

# --- Simulacion (III): Consola del Muro ---
preparar_muro = function(nombre_circuito, nombre_piloto, nombre_constructor,
  ↪ pos_salida = 1, forzar_lluvia = FALSE) {
  datos_pista = info_circuitos[info_circuitos$Circuito == nombre_circuito, ]

  penalizacion_tiempo = (pos_salida - 1) * 0.05
  agresividad_extra = (pos_salida - 1) * 0.1
  humedad_aplicada = datos_pista$Humedad_scaled_Media
  if(forzar_lluvia == TRUE) humedad_aplicada = 2.0

  muro = list(
    circuito = nombre_circuito, piloto = nombre_piloto, constructor =
      ↪ nombre_constructor,
    vueltas_totales = datos_pista$Vueltas_Totales, temp_pista_scaled =
      ↪ datos_pista$Temp_Pista_scaled_Media,
    humedad_scaled = humedad_aplicada, viento_scaled =
      ↪ datos_pista$Viento_scaled_Media,

```

```

tiempo_vuelta_base = 90.0, mejora_combustible = 0.06, evolucion_pista =
  ↪ 0.015,
posicion_salida = pos_salida, penalizacion_trafico = penalizacion_tiempo,
  ↪
agresividad_base_scaled = agresividad_extra, tiempo_pit_stop =
  ↪ datos_pista$Tiempo_PitStop_Medio,
penalizacion_outlap = 2.0, probabilidad_sc = 0.05, n_iteraciones = 10000
)
return(muro) }

carrera_actual = preparar_muro("Circuit de Barcelona-Catalunya", "Carlos
  ↪ Sainz", "Ferrari", 6, FALSE)

```

#### 0.5.4 Simulación (IV): Generador Heurístico

Se define la función `generar_top_estrategias()`, que propone un catálogo de estrategias de paradas en boxes (número de stints, compuestos asignados y vueltas objetivo de cada uno) en función de la posición de salida del piloto y de si las condiciones son de lluvia, replicando heurísticas reales de estrategia de carrera.

```

# --- Simulacion (IV): Generador Heuristico ---
generar_top_estrategias = function(muro) {
  vueltas = muro$vueltas_totales; posicion = muro$posicion_salida; humedad =
  ↪ muro$humedad_scaled
  estrategias = list()

  if (humedad > 1.2) {
    if (posicion >= 13) {
      estrategias[["Lluvia_Continua_Tardia"]] = data.frame(
        Stint = c(1, 2), Compuesto_Neumatico = c("WET", "INTERMEDIATE"),
        Vueltas_Objetivo = c(round(vueltas * 0.65), vueltas - round(vueltas *
          ↪ 0.65)))
    } else if (posicion >= 6 && posicion <= 12) {
      estrategias[["Lluvia_Continua_Anticipada"]] = data.frame(
        Stint = c(1, 2), Compuesto_Neumatico = c("WET", "INTERMEDIATE"),
        Vueltas_Objetivo = c(round(vueltas * 0.35), vueltas - round(vueltas *
          ↪ 0.35)))
    } else {
      estrategias[["Lluvia_Continua_Optima"]] = data.frame(
        Stint = c(1, 2), Compuesto_Neumatico = c("WET", "INTERMEDIATE"),
        Vueltas_Objetivo = c(round(vueltas * 0.45), vueltas - round(vueltas *
          ↪ 0.45))) }
  }
}

```



```

} else {
  if (posicion >= 13) {
    estrategias[["1_Parada_Invertida"]] = data.frame(
      Stint = c(1, 2), Compuesto_Neumatico = c("HARD", "SOFT"),
      Vueltas_Objetivo = c(round(vueltas * 0.65), vueltas - round(vueltas *
↪ 0.65)))
  } else if (posicion >= 6 && posicion <= 12) {
    estrategias[["2_Paradas_Undercut"]] = data.frame(
      Stint = c(1, 2, 3), Compuesto_Neumatico = c("SOFT", "HARD",
↪ "MEDIUM"),
      Vueltas_Objetivo = c(10, 33, 23))
    estrategias[["1_Parada_Undercut"]] = data.frame(
      Stint = c(1, 2), Compuesto_Neumatico = c("SOFT", "HARD"),
      Vueltas_Objetivo = c(16, 50))
    estrategias[["2_Paradas_Agresiva_Mid"]] = data.frame(
      Stint = c(1, 2, 3), Compuesto_Neumatico = c("MEDIUM", "SOFT",
↪ "SOFT"),
      Vueltas_Objetivo = c(30, 20, 16))
  } else {
    estrategias[["1_Parada_Optima"]] = data.frame(
      Stint = c(1, 2), Compuesto_Neumatico = c("MEDIUM", "HARD"),
      Vueltas_Objetivo = c(round(vueltas * 0.45), vueltas - round(vueltas *
↪ 0.45))) }
}
return(estrategias) }

estrategias_catologo = generar_top_estrategias(carrera_actual)

```

### 0.5.5 Simulación (V): Motor Estocástico de Caos

Se define la función `preparar_motor_estocastico()`, que extrae de los modelos mixtos los efectos aleatorios específicos del piloto y constructor de la carrera simulada, ajusta la desviación típica del ritmo si hay lluvia, incorpora el “bonus” mecánico de la escudería en boxes, y devuelve tres generadores de ruido aleatorio: para el ritmo de vuelta, para la vida útil del neumático y para el tiempo de parada (este último incluye un componente de error catastrófico poco probable, simulando un problema mecánico en el pit stop).

```

# --- Simulacion (V): Motor Estocastico de Caos ---
preparar_motor_estocastico = function(muro, mod_m1 = NULL, mod_m2 = NULL) {
  sigma_ritmo_base = if(is.null(mod_m1)) 1.2 else tryCatch(sigma(mod_m1),
↪ error = function(e) 1.2)

```

```

sigma_vida_base = if(is.null(mod_m2)) 3.5 else tryCatch(sigma(mod_m2),
↪ error = function(e) 3.5)

offset_ritmo_piloto = 0; offset_ritmo_coche = 0
offset_vida_piloto = 0; offset_vida_coche = 0

if(!is.null(mod_m1)) {
  if(muro$piloto %in% rownames(ranef(mod_m1)$Piloto)) offset_ritmo_piloto =
    ↪ ranef(mod_m1)$Piloto[muro$piloto, 1]
  if(muro$constructor %in% rownames(ranef(mod_m1)$Constructor))
    ↪ offset_ritmo_coche = ranef(mod_m1)$Constructor[muro$constructor, 1] }

if(!is.null(mod_m2)) {
  if(muro$piloto %in% rownames(ranef(mod_m2)$Piloto)) offset_vida_piloto =
    ↪ ranef(mod_m2)$Piloto[muro$piloto, 1]
  if(muro$constructor %in% rownames(ranef(mod_m2)$Constructor))
    ↪ offset_vida_coche = ranef(mod_m2)$Constructor[muro$constructor, 1] }

ventaja_ritmo_total = offset_ritmo_piloto + offset_ritmo_coche
ventaja_vida_total = offset_vida_piloto + offset_vida_coche

es_lluvia = muro$humedad_scaled > 1.2
sigma_ritmo_final = sigma_ritmo_base * ifelse(es_lluvia, 1.6, 1.0)

bonus_mecanicos = 0.0
if(exists("info_escuderias")) {
  fila_escuderia = info_escuderias[info_escuderias$Constructor ==
↪ muro$constructor, ]
  if(nrow(fila_escuderia) > 0 && !is.na(fila_escuderia$Bonus_Mecanicos[1]))
    ↪ bonus_mecanicos = fila_escuderia$Bonus_Mecanicos[1] }

motor = list(
  lanzar_dados_ritmo = function(n_iteraciones) {
    return(rnorm(n = n_iteraciones, mean = ventaja_ritmo_total, sd =
    ↪ sigma_ritmo_final)) },
  lanzar_dados_supervivencia = function(n_iteraciones) {
    return(rnorm(n = n_iteraciones, mean = ventaja_vida_total, sd =
    ↪ sigma_vida_base)) },
  lanzar_dados_pitstop = function(n_iteraciones) {
    ruido_estandar = rnorm(n_iteraciones, mean = bonus_mecanicos, sd = 0.4)
    error_critico = rbinom(n_iteraciones, 1, 0.04) * rexp(n_iteraciones,
    ↪ rate = 0.2)

```

```

        return(ruido_estandar + error_critico) }
    )
    return(motor) }

m1_activo = if(exists("modelo_lmer_m1")) modelo_lmer_m1 else NULL
m2_activo = if(exists("modelo_lmer_m2")) modelo_lmer_m2 else NULL
motor_caos = preparar_motor_estocastico(carrera_actual, mod_m1 = m1_activo,
↪   mod_m2 = m2_activo)

```

### 0.5.6 Simulación (VI): Integración de Monte Carlo Vectorizada

Para cada estrategia del catálogo se simulan, de forma vectorizada,  $N = 10.000$  carreras: se combina el tiempo base de vuelta, la mejora por consumo de combustible, la evolución de la pista, la degradación cuadrática propia de cada compuesto y el ruido estocástico de ritmo; se contabiliza el riesgo de pinchazo cuando la vida útil simulada del neumático es inferior a las vueltas objetivo; y se añade el coste de las paradas en boxes, con un posible descuento de tiempo si se despliega un coche de seguridad.

```

# --- Simulacion (VI): Integracion de Monte Carlo Vectorizada ---
N = carrera_actual$n_iteraciones
resultados_simulacion = list()

for (nombre_est in names(estrategias_catalogo)) {
  est = estrategias_catalogo[[nombre_est]]
  n_stints = nrow(est)

  tiempo_carrera_vector = rep(0, N)
  riesgo_pinchazos = rep(0, N)
  v_acumuladas = 0

  for (j in 1:n_stints) {
    compuesto = est$Compuesto_Neumatico[j]
    v_objetivo = est$Vueltas_Objetivo[j]

    ruido_ritmo = motor_caos$lanzar_dados_ritmo(N)
    ruido_vida = motor_caos$lanzar_dados_supervivencia(N)

    params = switch(compuesto,
                     "SOFT" = list(deg = 0.160, vida = 20),
                     "MEDIUM" = list(deg = 0.085, vida = 36),
                     "HARD" = list(deg = 0.045, vida = 54),

```

```

      "INTERMEDIATE" = list(deg = 0.120, vida = 30),
      "WET" = list(deg = 0.200, vida = 25))

vida_real = params$vida + ruido_vida
fallo_goma = v_objetivo > vida_real
riesgo_pinchazos = riesgo_pinchazos + fallo_goma
v_media_stint = v_acumuladas + (v_objetivo / 2)

tiempo_stint = (carrera_actual$tiempo_vuelta_base * v_objetivo) +
  (carrera_actual$penalizacion_trafico * v_objetivo) -
  (carrera_actual$mejora_combustible * v_media_stint * v_objetivo) -
  (carrera_actual$evolucion_pista * v_media_stint * v_objetivo) +
  (params$deg * (v_objetivo^2) / 2) +
  (ruido_ritmo * v_objetivo)

if (j > 1) tiempo_stint = tiempo_stint +
  ↪ carrera_actual$penalizacion_outlap
tiempo_stint = tiempo_stint + (fallo_goma * 45.0)

tiempo_carrera_vector = tiempo_carrera_vector + tiempo_stint
v_acumuladas = v_acumuladas + v_objetivo
}

coste_pit = (n_stints - 1) * (carrera_actual$tiempo_pit_stop +
↪ motor_caos$lanzar_dados_pitstop(N))
coste_pit_final = coste_pit - (rbinom(N, 1, carrera_actual$probabilidad_sc)
↪ * (carrera_actual$tiempo_pit_stop * 0.55))
tiempo_carrera_vector = tiempo_carrera_vector + coste_pit_final

resultados_simulacion[[nombre_est]] = data.frame(
  ID = 1:N, Tiempo_Total = tiempo_carrera_vector,
  Pinchazos = riesgo_pinchazos,
  SC_Desplegado = rbinom(N, 1, carrera_actual$probabilidad_sc)
) }

```

## 0.6 Análisis Estocástico de Resultados

### 0.6.1 Análisis Estocástico (I): Matriz de Decisión

Se combinan los resultados de todas las estrategias simuladas en una única tabla y se calculan, por estrategia, la media, la mediana y la desviación típica del tiempo total de carrera, junto

con la probabilidad empírica de reventón, formateando además el tiempo medio en formato hh:mm:ss para su lectura directa.

```
# --- Analisis Estocastico (I): Matriz de Decision ---
datos_completos = data.frame()
for (nombre in names(resultados_simulacion)) {
  df_temp = resultados_simulacion[[nombre]]
  df_temp$Estrategia = nombre
  datos_completos = rbind(datos_completos, df_temp) }

resumen_tiempos = aggregate(Tiempo_Total ~ Estrategia, data =
  ↪ datos_completos,
                           FUN = function(x) c(Media = mean(x), Mediana =
  ↪ median(x), Riesgo_Segundos = sd(x)))
resumen_tiempos = do.call(data.frame, resumen_tiempos)

resumen_pinchazos = aggregate(Pinchazos ~ Estrategia, data = datos_completos,
  ↪ sum)
resumen_tiempos$Probabilidad_Reventon_Pct = (resumen_pinchazos$Pinchazos / N)
  ↪ * 100
colnames(resumen_tiempos) = c("Estrategia", "Media_Segundos",
  ↪ "Mediana_Segundos", "Riesgo_Segundos", "Prob_Reventon_Pct")

segundos_a_hora = function(segundos) {
  h = floor(segundos / 3600)
  m = floor((segundos %% 3600) / 60)
  s = segundos %% 60
  return(sprintf("%02d:%02d:%06.3f", h, m, s)) }

resumen_tiempos$Tiempo_Medio_Format = sapply(resumen_tiempos$Media_Segundos,
  ↪ segundos_a_hora)
tabla_tfg = resumen_tiempos[, c("Estrategia", "Media_Segundos",
  ↪ "Tiempo_Medio_Format", "Prob_Reventon_Pct")]
```

### 0.6.2 Análisis Estocástico (II): Representación Densidad KDE

Se representa la densidad de probabilidad (KDE) del tiempo total de carrera para cada una de las estrategias simuladas, lo que permite comparar visualmente su rendimiento esperado y su grado de dispersión o riesgo.

```

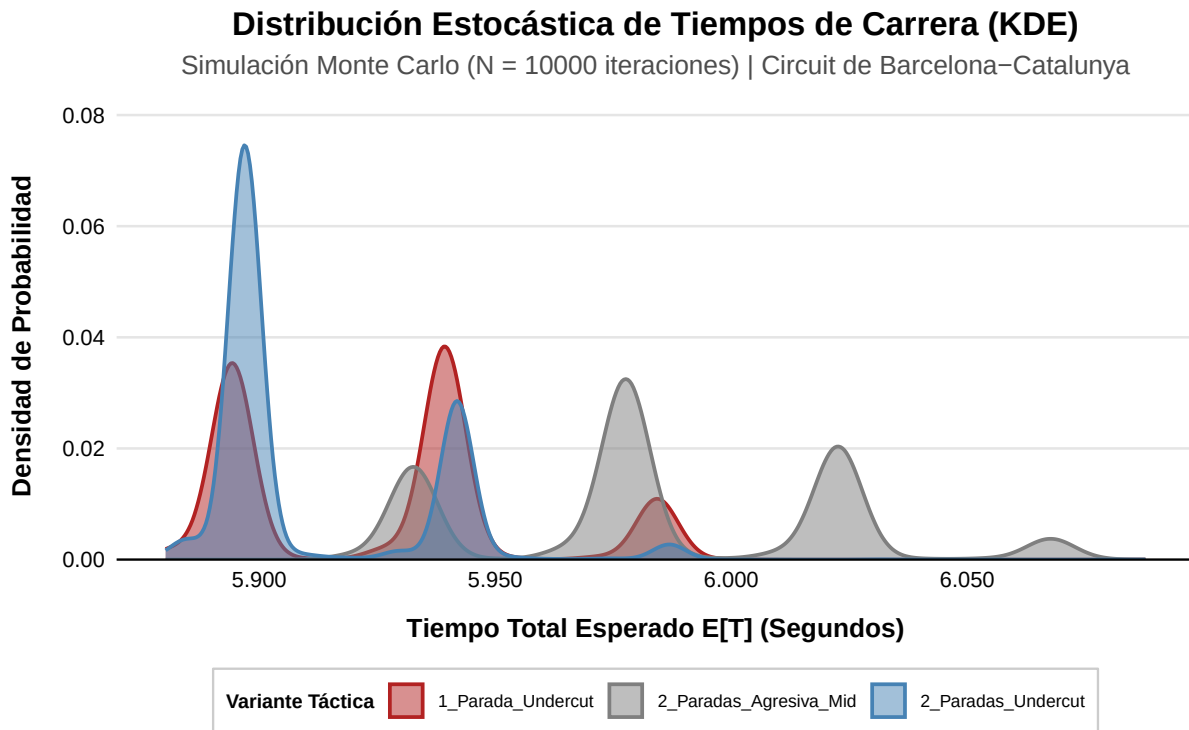
# --- Analisis Estocastico (II): Representacion Densidad KDE ---
library(ggplot2)

colores_tfg = c("1_Parada_Undercut" = "firebrick",
                "2_Paradas_Agresiva_Mid" = "grey50",
                "2_Paradas_Undercut" = "steelblue")

grafica_montecarlo = ggplot(datos_completos, aes(x = Tiempo_Total, fill =
  ↳ Estrategia, color = Estrategia)) +
  geom_density(alpha = 0.5, linewidth = 0.8) +
  scale_fill_manual(values = colores_tfg) +
  scale_color_manual(values = colores_tfg) +
  scale_x_continuous(labels = scales::comma_format(big.mark = ".",
  ↳ decimal.mark = ",")) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.08))) +
  labs(title = "Distribución Estocástica de Tiempos de Carrera (KDE)",
       subtitle = paste0("Simulación Monte Carlo (N = ", N, " iteraciones) |
  ↳ ", carrera_actual$circuito),
       x = "Tiempo Total Esperado E[T] (Segundos)", y = "Densidad de
  ↳ Probabilidad",
       fill = "Variante Táctica", color = "Variante Táctica") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom",
        legend.background = element_rect(fill = "white", color = "gray80",
        ↳ linewidth = 0.5),
        legend.title = element_text(face = "bold", size = 9),
        legend.text = element_text(size = 8),
        plot.title = element_text(face = "bold", size = 15, hjust = 0.5),
        plot.subtitle = element_text(size = 12, hjust = 0.5, color =
        ↳ "gray30", margin = margin(b = 15)),
        axis.title.x = element_text(face = "bold", margin = margin(t = 12)),
        axis.title.y = element_text(face = "bold", margin = margin(r = 12)),
        axis.text = element_text(color = "black"),
        axis.line.x = element_line(color = "black", linewidth = 0.5),
        panel.grid.major.y = element_line(color = "gray90"),
        panel.grid.major.x = element_blank(),
        panel.grid.minor = element_blank(),
        plot.margin = margin(t = 15, r = 20, b = 15, l = 15))

print(grafica_montecarlo)

```



### 0.6.3 Análisis Estocástico (III): Gráfico de Convergencia Asintótica

Se calcula la media acumulada del tiempo de carrera a medida que avanza el número de iteraciones de la simulación y se representa gráficamente, evidenciando la convergencia del estimador de Monte Carlo conforme a la Ley de los Grandes Números.

```
# --- Analisis Estocastico (III): Grafico de Convergencia Asintotica ---
datos_completos$Media_Acumulada = ave(datos_completos$Tiempo_Total,
  ↪ datos_completos$Estrategia,
                                     FUN = function(x) cumsum(x) /
                                     ↪ seq_along(x))

grafica_convergencia = ggplot(datos_completos, aes(x = ID, y =
  ↪ Media_Acumulada, color = Estrategia)) +
  geom_line(linewidth = 0.8) +
  scale_color_manual(values = colores_tfg) +
  scale_x_continuous(labels = scales::comma_format(big.mark = ".",
  ↪ decimal.mark = ",")) +
  labs(title = "Convergencia del Estimador de Monte Carlo",
       subtitle = paste0("Demostración de la Ley de los Grandes Números
  ↪ (Estabilidad con N = ", N, ")"),
```

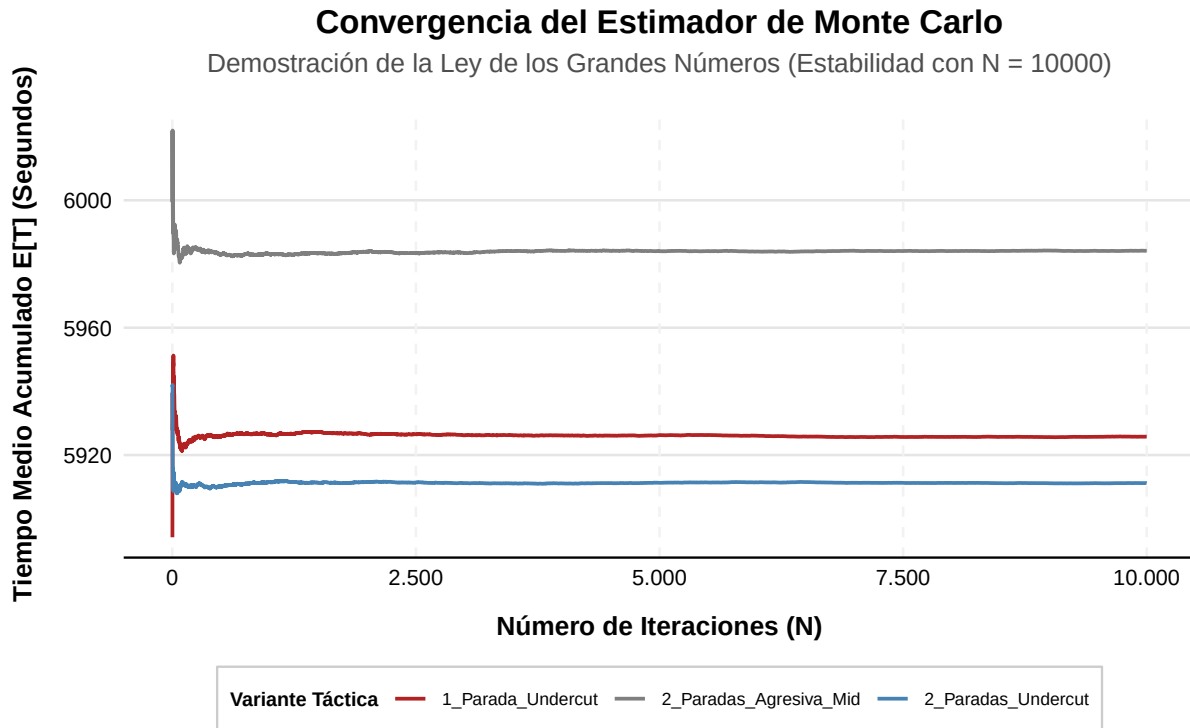
```

x = "Número de Iteraciones (N)", y = "Tiempo Medio Acumulado E[T]
    ↪ (Segundos)", color = "Variante Táctica") +
theme_minimal(base_size = 12) +
theme(legend.position = "bottom",
      legend.background = element_rect(fill = "white", color = "gray80",
    ↪ linewidth = 0.5),
      legend.title = element_text(face = "bold", size = 9),
      legend.text = element_text(size = 8),
      plot.title = element_text(face = "bold", size = 15, hjust = 0.5),
      plot.subtitle = element_text(size = 12, hjust = 0.5, color =
    ↪ "gray30", margin = margin(b = 15)),
      axis.title.x = element_text(face = "bold", margin = margin(t = 12)),
      axis.title.y = element_text(face = "bold", margin = margin(r = 12)),
      axis.text = element_text(color = "black"),
      axis.line.x = element_line(color = "black", linewidth = 0.5),
      panel.grid.major.y = element_line(color = "gray90"),
      panel.grid.major.x = element_line(color = "gray95", linetype =
    ↪ "dashed"),
      panel.grid.minor = element_blank(),
      plot.margin = margin(t = 15, r = 20, b = 15, l = 15))

print(grafica_convergencia)

```





## 0.7 Optimización Prescriptiva

### 0.7.1 Optimización Prescriptiva (I): Heurística de Transferencia

Para cada estrategia cuya probabilidad de reventón supera el umbral de riesgo aceptable, se identifica el stint con menor margen de seguridad (más arriesgado) y el de mayor margen (más seguro), y se transfiere un 20% de las vueltas del primero al segundo, generando una versión “corregida” de la estrategia con un perfil de riesgo más equilibrado.

```
# --- Optimizacion Prescriptiva (I): Heuristica de Transferencia ---
umbral_riesgo_aceptable = 15.0
estrategias_corregidas = list()

calcular_margen = function(compuesto, vueltas) {
  vida_base = switch(compuesto, "SOFT" = 20, "MEDIUM" = 36, "HARD" = 54,
  ↪ "INTERMEDIATE" = 30, "WET" = 25)
  return(vida_base - vueltas) }

for (nombre in names(estrategias_catalogo)) {
  est_actual = estrategias_catalogo[[nombre]]
```

```

riesgo_actual = tabla_tfg$Prob_Reventon_Pct[tabla_tfg$Estrategia == nombre]

if (riesgo_actual <= umbral_riesgo_aceptable) {
  estrategias_corregidas[[nombre]] = est_actual
  next }

margenes = sapply(1:nrow(est_actual), function(i)
↪   calcular_margen(est_actual$Compuesto_Neumatico[i],
↪   est_actual$Vueltas_Objetivo[i]))

stint_mas_peligroso = which.min(margenes)

# Forzado algoritmico para evitar superposicion iterativa
margenes_seguros = margenes
margenes_seguros[stint_mas_peligroso] = -Inf
stint_mas_seguro = which.max(margenes_seguros)

vueltas_a_transferir = max(1,
↪   round(est_actual$Vueltas_Objetivo[stint_mas_peligroso] * 0.20))

est_actual$Vueltas_Objetivo[stint_mas_peligroso] =
↪   est_actual$Vueltas_Objetivo[stint_mas_peligroso] - vueltas_a_transferir
est_actual$Vueltas_Objetivo[stint_mas_seguro] =
↪   est_actual$Vueltas_Objetivo[stint_mas_seguro] + vueltas_a_transferir

nuevo_nombre = paste0(nombre, "_ (Corregida)")
estrategias_corregidas[[nuevo_nombre]] = est_actual }

```

## 0.7.2 Optimización Prescriptiva (II): Evaluación Multicriterio

Se vuelven a simular, con una muestra reducida ( $N = 5.000$  iteraciones, por eficiencia computacional) todas las estrategias corregidas, se calcula su tiempo medio y su riesgo de reventón, y se selecciona como estrategia “ganadora” aquella con menor tiempo medio entre las que cumplen el criterio de riesgo aceptable.

```

# --- Optimizacion Prescriptiva (II): Evaluacion Multicriterio ---
N_fast = 5000
resultados_finales = data.frame()

for (nombre in names(estrategias_corregidas)) {
  est = estrategias_corregidas[[nombre]]

```

```

n_stints = nrow(est)
tiempo_total_vector = rep(0, N_fast)
riesgo_pinchazos = rep(0, N_fast)
v_acumuladas = 0

for (j in 1:n_stints) {
  comp = est$Compuesto_Neumatico[j]; v_obj = est$Vueltas_Objetivo[j]
  r_ritmo = motor_caos$lanzar_dados_ritmo(N_fast)
  r_vida = motor_caos$lanzar_dados_supervivencia(N_fast)

  params = switch(comp, "SOFT"=list(deg=0.160, vida=20),
    ↪ "MEDIUM"=list(deg=0.085, vida=36),
      "HARD"=list(deg=0.045, vida=54),
        ↪ "INTERMEDIATE"=list(deg=0.120, vida=30),
          "WET"=list(deg=0.200, vida=25))
  vida_real = params$vida + r_vida; fallo = v_obj > vida_real
  riesgo_pinchazos = riesgo_pinchazos + fallo
  v_med = v_acumuladas + (v_obj / 2)

  t_stint = (carrera_actual$tiempo_vuelta_base * v_obj) +
    (carrera_actual$penalizacion_trafico * v_obj) -
    (carrera_actual$mejora_combustible * v_med * v_obj) -
    (carrera_actual$evolucion_pista * v_med * v_obj) +
    (params$deg * (v_obj^2) / 2) + (r_ritmo * v_obj) + (fallo * 45.0)

  if (j > 1) t_stint = t_stint + carrera_actual$penalizacion_outlap
  tiempo_total_vector = tiempo_total_vector + t_stint
  v_acumuladas = v_acumuladas + v_obj
}

coste_pit = (n_stints - 1) * (carrera_actual$tiempo_pit_stop +
  ↪ motor_caos$lanzar_dados_pitstop(N_fast))
coste_pit_final = coste_pit - (rbinom(N_fast, 1,
  ↪ carrera_actual$probabilidad_sc) * (carrera_actual$tiempo_pit_stop *
  ↪ 0.55))
tiempo_total_vector = tiempo_total_vector + coste_pit_final

resultados_finales = rbind(resultados_finales, data.frame(
  Estrategia = nombre, Tiempo_Medio = mean(tiempo_total_vector),
  Riesgo_Pct = (sum(riesgo_pinchazos) / N_fast) * 100)) }

estrategias_viables = resultados_finales[resultados_finales$Riesgo_Pct <=
  ↪ umbral_riesgo_aceptable + 15, ]

```

```
if(nrow(estراتيجias_viables) == 0) estrategias_viables = resultados_finales
ganadora = estrategias_viables[which.min(estراتيجias_viables$Tiempo_Medio), ]
```

### 0.7.3 Optimización Prescriptiva (III): Frontera de Eficiencia de Pareto

Se combinan los resultados originales y los optimizados en un único gráfico de dispersión (tiempo total esperado frente a riesgo de reventón), que muestra de forma visual la mejora conseguida por la corrección heurística respecto al umbral de riesgo aceptable, marcado mediante una línea de referencia.

```
# --- Optimizacion Prescriptiva (III): Frontera de Eficiencia de Pareto ---
library(ggplot2)

df_originales = data.frame(
  Estrategia = tabla_tfg$Estrategia,
  Tiempo_Medio = sapply(tabla_tfg$Estrategia, function(nombre)
    ↪ mean(resultados_simulacion[[nombre]]$Tiempo_Total)),
  Riesgo_Pct = tabla_tfg$Prob_Reventon_Pct,
  Fase = "Original (Baseline)")

df_optimizadas = data.frame(
  Estrategia = resultados_finales$Estrategia,
  Tiempo_Medio = resultados_finales$Tiempo_Medio,
  Riesgo_Pct = resultados_finales$Riesgo_Pct,
  Fase = "Optimizada (Ingeniero Virtual)")

datos_pareto = rbind(df_originales, df_optimizadas)
datos_pareto$Fase = factor(datos_pareto$Fase, levels = c("Original
  ↪ (Baseline)", "Optimizada (Ingeniero Virtual)"))
colores_pareto = c("Original (Baseline)" = "#c0392b", "Optimizada (Ingeniero
  ↪ Virtual)" = "#27ae60")

grafica_pareto = ggplot(datos_pareto, aes(x = Tiempo_Medio, y = Riesgo_Pct,
  ↪ color = Fase, shape = Fase)) +
  geom_point(size = 5, alpha = 0.85) +
  geom_hline(yintercept = umbral_riesgo_aceptable, linetype = "dashed", color
    ↪ = "#c0392b", linewidth = 0.8) +
  scale_color_manual(values = colores_pareto) +
  scale_shape_manual(values = c("Original (Baseline)" = 17, "Optimizada
    ↪ (Ingeniero Virtual)" = 16)) +
  scale_x_continuous(labels = scales::comma_format(big.mark = ".",
    ↪ decimal.mark = ",")) +
```

```

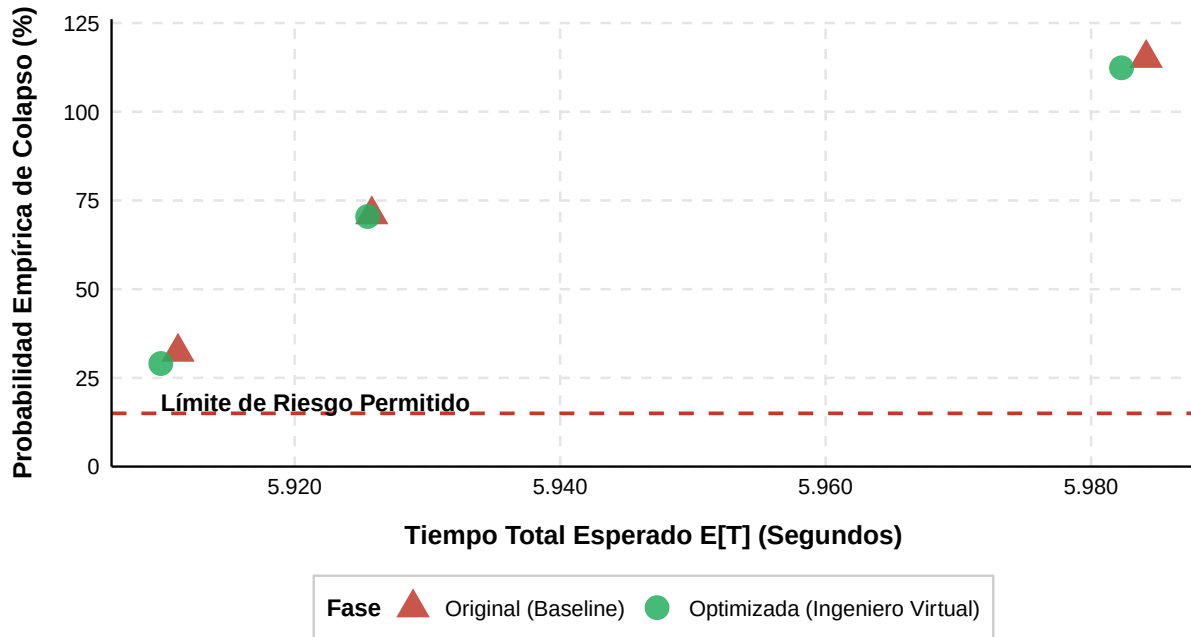
scale_y_continuous(limits = c(0, max(datos_pareto$Riesgo_Pct) + 5), expand
  ↪ = expansion(mult = c(0, 0.05))) +
labs(title = "Frontera de Eficiencia de Pareto: Tiempo vs Riesgo
  ↪ Estocástico",
  subtitle = paste("Evaluación del catálogo táctico -",
    ↪ carrera_actual$piloto),
  x = "Tiempo Total Esperado E[T] (Segundos)", y = "Probabilidad
    ↪ Empírica de Colapso (%)",
  color = "Fase", shape = "Fase") +
theme_minimal(base_size = 12) +
theme(legend.position = "bottom",
  legend.background = element_rect(fill = "white", color = "gray80",
    ↪ linewidth = 0.5),
  legend.title = element_text(face = "bold", size = 11),
  legend.text = element_text(size = 10),
  plot.title = element_text(face = "bold", size = 15, hjust = 0.5),
  plot.subtitle = element_text(size = 12, hjust = 0.5, color =
    ↪ "gray30", margin = margin(b = 15)),
  axis.title.x = element_text(face = "bold", margin = margin(t = 12)),
  axis.title.y = element_text(face = "bold", margin = margin(r = 12)),
  axis.text = element_text(color = "black"),
  axis.line.x = element_line(color = "black", linewidth = 0.5),
  axis.line.y = element_line(color = "black", linewidth = 0.5),
  panel.grid.major = element_line(color = "gray90", linetype =
    ↪ "dashed"),
  panel.grid.minor = element_blank(),
  plot.margin = margin(t = 15, r = 20, b = 15, l = 15)) +
annotate("text", x = min(datos_pareto$Tiempo_Medio), y =
  ↪ umbral_riesgo_aceptable + 3,
  label = "Límite de Riesgo Permitido", color = "black", fontface =
    ↪ "bold", hjust = 0)

print(grafica_pareto)

```

## Frontera de Eficiencia de Pareto: Tiempo vs Riesgo Estocástico

Evaluación del catálogo táctico – Carlos Sainz



## 0.8 Auditoría Final del Modelo

### 0.8.1 Auditoría Final (I): Simulación Extendida del Optimo (N=10.000)

Se vuelve a simular, ya con la muestra completa ( $N = 10.000$  iteraciones), tanto la estrategia ganadora corregida como su versión original sin corregir, preparando los datos para la comparación final de auditoría.

```
# --- Auditoria Final (I): Simulacion Extendida del Optimo (N=10.000) ---
nombre_ganadora_corregida = ganadora$Estrategia
nombre_original = sub("_\\(Corregida\\)", "", nombre_ganadora_corregida)

est_final = estrategias_corregidas[[nombre_ganadora_corregida]]
n_stints = nrow(est_final)
tiempo_total_vector_final = rep(0, N)
riesgo_pinchazos_final = rep(0, N)
v_acumuladas = 0

for (j in 1:n_stints) {
  comp = est_final$Compuesto_Neumatico[j]; v_obj =
  ↪ est_final$Vueltas_Objetivo[j]
```

```

r_ritmo = motor_caos$lanzar_dados_ritmo(N); r_vida =
↪ motor_caos$lanzar_dados_supervivencia(N)

params = switch(comp, "SOFT"=list(deg=0.160, vida=20),
↪ "MEDIUM"=list(deg=0.085, vida=36),
      "HARD"=list(deg=0.045, vida=54),
      ↪ "INTERMEDIATE"=list(deg=0.120, vida=30),
      "WET"=list(deg=0.200, vida=25))
vida_real = params$vida + r_vida; fallo = v_obj > vida_real
riesgo_pinchazos_final = riesgo_pinchazos_final + fallo
v_med = v_acumuladas + (v_obj / 2)

t_stint = (carrera_actual$tiempo_vuelta_base * v_obj) +
  (carrera_actual$penalizacion_trafico * v_obj) -
  (carrera_actual$mejora_combustible * v_med * v_obj) -
  (carrera_actual$evolucion_pista * v_med * v_obj) +
  (params$deg * (v_obj^2) / 2) + (r_ritmo * v_obj) + (fallo * 45.0)

if (j > 1) t_stint = t_stint + carrera_actual$penalizacion_outlap
tiempo_total_vector_final = tiempo_total_vector_final + t_stint
v_acumuladas = v_acumuladas + v_obj
}

coste_pit = (n_stints - 1) * (carrera_actual$tiempo_pit_stop +
↪ motor_caos$lanzar_dados_pitstop(N))
coste_pit_final = coste_pit - (rbinom(N, 1, carrera_actual$probabilidad_sc)
↪ * (carrera_actual$tiempo_pit_stop * 0.55))
tiempo_total_vector_final = tiempo_total_vector_final + coste_pit_final

datos_originales = resultados_simulacion[[nombre_original]]
df_corregida = data.frame(ID = 1:N, Tiempo_Total = tiempo_total_vector_final,
↪
      Pinchazos = riesgo_pinchazos_final, Estrategia =
      ↪ "Óptima Corregida (Segura)")
df_original = data.frame(ID = 1:N, Tiempo_Total =
↪ datos_originales$Tiempo_Total,
      Pinchazos = datos_originales$Pinchazos, Estrategia =
      ↪ "Original Base (Riesgo)")

datos_comparativa = rbind(df_original, df_corregida)
datos_comparativa$Estrategia = factor(datos_comparativa$Estrategia, levels =
↪ c("Original Base (Riesgo)", "Óptima Corregida (Segura)"))

```

## 0.8.2 Auditoría Final (II): Gráfico Comparativo de Reducción de Varianza

Se representan las densidades KDE de la estrategia original y de la estrategia corregida una junto a la otra, mostrando visualmente la reducción de riesgo y de varianza conseguida gracias al proceso de optimización prescriptiva.

```
# --- Auditoria Final (II): Grafico Comparativo de Reduccion de Varianza ---
library(ggplot2)

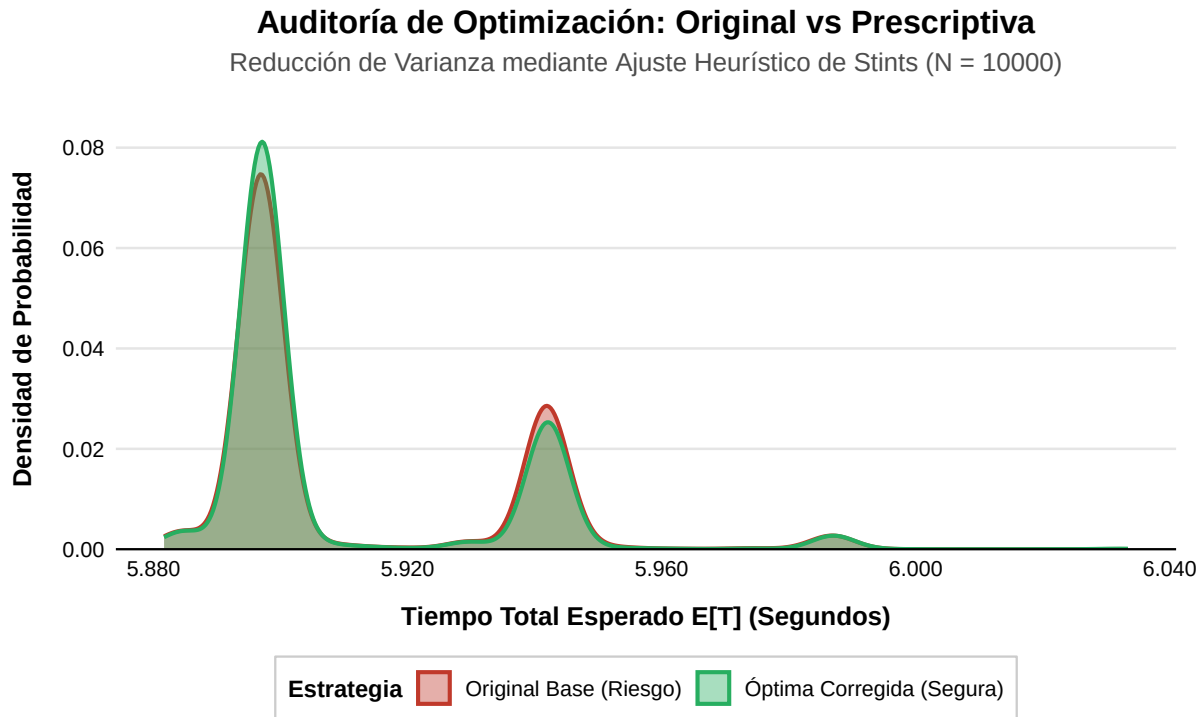
colores_auditoria = c("Original Base (Riesgo)" = "#c0392b", "Óptima Corregida
  ↪ (Segura)" = "#27ae60")

grafica_auditoria = ggplot(datos_comparativa, aes(x = Tiempo_Total, fill =
  ↪ Estrategia, color = Estrategia)) +
  geom_density(alpha = 0.4, linewidth = 0.9) +
  scale_fill_manual(values = colores_auditoria) +
  scale_color_manual(values = colores_auditoria) +
  scale_x_continuous(labels = scales::comma_format(big.mark = ".",
  ↪ decimal.mark = ",")) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.08))) +
  theme_minimal(base_size = 12) +
  labs(title = "Auditoría de Optimización: Original vs Prescriptiva",
    subtitle = paste0("Reducción de Varianza mediante Ajuste Heurístico de
  ↪ Stints (N = ", N, ")"),
    x = "Tiempo Total Esperado E[T] (Segundos)", y = "Densidad de
  ↪ Probabilidad",
    fill = "Estrategia", color = "Estrategia") +
  theme(legend.position = "bottom",
    legend.background = element_rect(fill = "white", color = "gray80",
  ↪ linewidth = 0.5),
    legend.title = element_text(face = "bold", size = 11),
    legend.text = element_text(size = 10),
    plot.title = element_text(face = "bold", size = 15, hjust = 0.5),
    plot.subtitle = element_text(size = 12, hjust = 0.5, color =
  ↪ "gray30", margin = margin(b = 15)),
    axis.title.x = element_text(face = "bold", margin = margin(t = 12)),
    axis.title.y = element_text(face = "bold", margin = margin(r = 12)),
    axis.text = element_text(color = "black"),
    axis.line.x = element_line(color = "black", linewidth = 0.5),
    panel.grid.major.y = element_line(color = "gray90"),
    panel.grid.major.x = element_blank(),
    panel.grid.minor = element_blank(),
```



```
plot.margin = margin(t = 15, r = 20, b = 15, l = 15))

print(grafica_auditoria)
```



### 0.8.3 Auditoría Final (III): Diagrama de Prescripción Táctica (Gantt)

Se representa la estrategia ganadora final como un diagrama de barras apiladas de tipo Gantt, mostrando el número de vueltas asignado a cada compuesto de neumático en cada stint, e incluyendo en el subtítulo el tiempo total esperado de la estrategia.

```
# --- Auditoria Final (III): Diagrama de Prescripcion Tactica (Gantt) ---
library(ggplot2)

est_ganadora_df = estrategias_corregidas[[as.character(ganadora$Estrategia)]]
colores_pirelli = c("SOFT" = "#e74c3c", "MEDIUM" = "#f1c40f", "HARD" =
  ↪ "#fbbf6b",
                    "INTERMEDIATE" = "#2ecc71", "WET" = "#3498db")

# Solucion a bug de codificacion de acentos en el eje X de ggplot2
est_ganadora_df$Stint_Factor = factor(est_ganadora_df$Stint, levels =
  ↪ rev(est_ganadora_df$Stint))
```

```

est_ganadora_df$Etiqueta_X = "Estrategia Optima"

grafica_stints = ggplot(est_ganadora_df, aes(x = Etiqueta_X, y =
  ↪ Vueltas_Objetivo, fill = Compuesto_Neumatico, group = Stint_Factor)) +
  geom_bar(stat = "identity", color = "black", linewidth = 0.6, width = 0.35)
  ↪ +
  geom_text(aes(label = ifelse(Vueltas_Objetivo <= 10,
                              paste(Vueltas_Objetivo, "v."),
                              paste(Vueltas_Objetivo, "vueltas"))),
            position = position_stack(vjust = 0.5), color = "black", fontface
  ↪ = "bold", size = 3.8) +
  coord_flip() +
  scale_fill_manual(values = colores_pirelli) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.05))) +
  labs(title = "Prescripción Táctica Final (Veredicto del Muro)",
       subtitle = paste0("Táctica: ", as.character(ganadora$Estrategia),
                          " | Tiempo Esperado E[T]: ", segundos_a_hora(
  ↪ as.numeric(ganadora$Tiempo_Medio))),
  ↪
       x = "", y = "Número de Vueltas Acumuladas", fill = "Compuesto
  ↪ Asignado") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom",
        legend.background = element_rect(fill = "white", color = "gray80",
  ↪ linewidth = 0.5),
        legend.title = element_text(face = "bold", size = 11),
        legend.text = element_text(size = 10),
        plot.title = element_text(face = "bold", size = 15, hjust = 0.5),
        plot.subtitle = element_text(size = 12, hjust = 0.5, color =
  ↪ "gray30", margin = margin(b = 20)),
        axis.title.x = element_text(face = "bold", margin = margin(t = 12)),
        axis.text.y = element_blank(),
        axis.ticks.y = element_blank(),
        axis.line.x = element_line(color = "black", linewidth = 0.5),
        axis.text.x = element_text(color = "black"),
        panel.grid.major.y = element_blank(),
        panel.grid.major.x = element_line(color = "gray90", linetype =
  ↪ "dashed"),
        panel.grid.minor = element_blank(),
        plot.margin = margin(t = 15, r = 20, b = 15, l = 15))

print(grafica_stints)

```

## Prescripción Táctica Final (Veredicto del Muro)

Táctica: 2\_Paradas\_Undercut\_(Corregida) | Tiempo Esperado  $E[T]$ : 01:38:29.915

